EN4554 Assignment 2 — Convolutional Neural Networks

Name: Savindu Dilshan Wickramasinghe **Index No.:** 220701X

GitHub repository: https://github.com/Savidilsh/EN4554-Assignment2-CNNs

Acknowledgement of AI tool use. Claude (Anthropic, via Claude Code) was used as a coding assistant to help write and debug the code in this notebook. I have read, run and understood all of the code and the interpretations below.

All experiments use PyTorch and CIFAR-10. The 50k training images are split once (fixed seed) into 45k train / 5k validation, every image is normalized per channel with the mean/std of the 45k training images, and the test set is touched exactly once, at the very end of Task 4.

Contents

0. Setup: data split, normalization, GPU batching and augmentation, training loop
1. Task 1: Backpropagation (manual layers, gradient checks, MLP training, gradient flow)
2. Task 2: Optimizers (custom SGD-momentum / Adam, trajectories, learning-rate sensitivity)
3. Task 3: Regularization (5k images) and plain vs residual networks (depth 8 / 20 / 56)
4. Task 4: Best CNN with ≤ 500k parameters and ≤ 20 epochs, ablation, single test evaluation

Each sub-task has: a short description of the method and settings → code → outputs (logs, tables, plots) → an **Interpretation** of the measured results.

0. Setup

Environment. The notebook runs on a Colab T4 or on a laptop GPU (the outputs below were produced on an NVIDIA GTX 1650, 4 GB). On Colab, results are saved to Google Drive so that a disconnected session can be resumed.

Speed. The whole normalized dataset is kept on the GPU, and mini-batching, random crop/flip, cutout and mixup are all done with tensor operations on the GPU, avoiding the CPU `DataLoader` bottleneck. Mixed precision (fp16) is used only on GPUs with tensor cores (e.g. T4); on the GTX 1650 fp16 is ~3.5× slower than fp32, so fp32 is used.

Resumability. Every training run is cached as JSON (history + its printed training log) in `results/`. Re-running the notebook re-uses finished runs and prints their saved logs instead of training again.

In [1]:

```
import copy, json, math, os, random, time, warnings

import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
import torch
import torch.nn as nn
import torch.nn.functional as F
import torchvision
from IPython.display import display

warnings.filterwarnings("ignore", message=".*lr_scheduler.step.*")
pd.set_option("display.precision", 4)

SEED = 0
QUICK = os.environ.get("A2_QUICK", "0") == "1"  # smoke-test mode: a few batches
per run

try:  # on Colab, keep data/results on Google Drive
    from google.colab import drive
    drive.mount("/content/drive")
    BASE = "/content/drive/MyDrive/EN4554_A2"
except ImportError:
    BASE = "."
DATA_DIR = os.path.join(BASE, "data")
RESULTS = os.path.join(BASE, "results_quick" if QUICK else "results")
FIGS = os.path.join(BASE, "figures_quick" if QUICK else "figures")
CKPT = os.path.join(BASE, "checkpoints")
for d in (DATA_DIR, RESULTS, FIGS, CKPT):
    os.makedirs(d, exist_ok=True)

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
# fp16 autocast only pays off on GPUs with tensor cores (compute capability >=
7.0, not the GTX 16xx series)
AMP = (device.type == "cuda" and torch.cuda.get_device_capability()[0] >= 7
       and "GTX" not in torch.cuda.get_device_name(0))
torch.backends.cudnn.benchmark = True


def set_seed(seed=SEED):
    random.seed(seed); np.random.seed(seed)
    torch.manual_seed(seed); torch.cuda.manual_seed_all(seed)


def savefig(name):
    plt.savefig(f"{FIGS}/{name}.png", dpi=130, bbox_inches="tight")


_LOG_CAPTURE = None


def log(msg):
    """Print a training-log line (also kept with the cached run and appended to
RESULTS/progress.log)."""
    print(msg, flush=True)
```

```
    if _LOG_CAPTURE is not None:
        _LOG_CAPTURE.append(msg)
    with open(f"{RESULTS}/progress.log", "a") as f:
        f.write(time.strftime("%H:%M:%S ") + msg + "\n")


def cached(name, fn):
    """Run fn() once and cache its result (a dict) with its training log in
RESULTS/name.json.
    If the run was already done, load it and print the saved training log instead
of retraining."""
    global _LOG_CAPTURE
    path = f"{RESULTS}/{name}.json"
    if os.path.exists(path):
        with open(path) as f:
            out = json.load(f)
        print(f"[{name}: loaded saved run; its training log was:]")
        for line in out.get("_log", []):
            print("  " + line)
        return out
    _LOG_CAPTURE = []
    try:
        out = fn()
        out["_log"] = _LOG_CAPTURE
    finally:
        _LOG_CAPTURE = None
    with open(path, "w") as f:
        json.dump(out, f)
    return out


def epochs(e):
    return 1 if QUICK else e


set_seed()
print("device:", device, torch.cuda.get_device_name(0) if device.type == "cuda"
else "", "| mixed precision:", AMP)
print("torch", torch.__version__, "| results in", os.path.abspath(RESULTS), "|
QUICK =", QUICK)
```

```
device: cuda NVIDIA GeForce GTX 1650 | mixed precision: False
torch 2.11.0+cu128 | results in D:\dev\repos\Deep Vision FT\Assignment 2\results |
QUICK = False
```

Data: 45k / 5k split and per-channel normalization with training statistics

In [2]:

```
train_set = torchvision.datasets.CIFAR10(DATA_DIR, train=True, download=True)
test_set = torchvision.datasets.CIFAR10(DATA_DIR, train=False, download=True)
CLASSES = train_set.classes

X_all = torch.from_numpy(train_set.data).permute(0, 3, 1, 2).contiguous()  #
uint8, N x 3 x 32 x 32
y_all = torch.tensor(train_set.targets)
```

```
perm = torch.randperm(len(y_all), generator=torch.Generator().manual_seed(SEED))
tr_idx, va_idx = perm[:45000], perm[45000:]

# per-channel statistics of the 45k training images only
MEAN = torch.stack([X_all[tr_idx, c].float().div(255).mean() for c in range(3)])
STD = torch.stack([X_all[tr_idx, c].float().div(255).std() for c in range(3)])


def normalize(X):
    return (X.float().div(255) - MEAN[:, None, None]) / STD[:, None, None]


X_tr, y_tr = normalize(X_all[tr_idx]).to(device), y_all[tr_idx].to(device)
X_va, y_va = normalize(X_all[va_idx]).to(device), y_all[va_idx].to(device)
# the test set stays on the CPU and is only used once, in Task 4
X_te = normalize(torch.from_numpy(test_set.data).permute(0, 3, 1, 2))
y_te = torch.tensor(test_set.targets)

print("train", tuple(X_tr.shape), "val", tuple(X_va.shape), "test",
tuple(X_te.shape))
print("per-channel mean (R, G, B):", MEAN.numpy().round(4), " std:",
STD.numpy().round(4))
display(pd.DataFrame({"train (45k)": torch.bincount(y_tr.cpu()).tolist(),
                      "validation (5k)": torch.bincount(y_va.cpu()).tolist()},
index=CLASSES).T)

# a few (un-normalized) training images
fig, axes = plt.subplots(2, 10, figsize=(15, 3.4))
for k, ax in enumerate(axes.flat):
    i = int(tr_idx[k])
    ax.imshow(train_set.data[i]); ax.set_title(CLASSES[train_set.targets[i]],
fontsize=8); ax.axis("off")
plt.suptitle("Sample CIFAR-10 training images"); plt.tight_layout(); plt.show()
```

```
train (45000, 3, 32, 32) val (5000, 3, 32, 32) test (10000, 3, 32, 32)
per-channel mean (R, G, B): [0.4912 0.482  0.4464]  std: [0.2472 0.2436 0.2617]
```

	airplane	automobile	bird	cat	deer	dog	frog	horse	ship	truck
train (45k)	4518	4492	4506	4496	4468	4493	4498	4494	4502	4533
validation (5k)	482	508	494	504	532	507	502	506	498	467



Batching, GPU augmentation (flip + crop, cutout, mixup) and a generic training loop

In [3]:
```
def batches(X, y, bs, shuffle=True):
    n = len(y)
```

```
    idx = torch.randperm(n, device=X.device) if shuffle else torch.arange(n,
device=X.device)
    for k, i in enumerate(range(0, n, bs)):
        if QUICK and k >= 3:
            break
        j = idx[i:i + bs]
        yield X[j], y[j]


def flip_crop(x, pad=4):
    """Random horizontal flip + random 32x32 crop from a zero-padded (= mean-
padded) image."""
    N, C, H, W = x.shape
    flip = torch.rand(N, device=x.device) < 0.5
    x = torch.where(flip[:, None, None, None], x.flip(3), x)
    xp = F.pad(x, (pad, pad, pad, pad))
    i = torch.randint(0, 2 * pad + 1, (N,), device=x.device)
    j = torch.randint(0, 2 * pad + 1, (N,), device=x.device)
    rows = (i[:, None] + torch.arange(H, device=x.device))[:, None, :, None]
    cols = (j[:, None] + torch.arange(W, device=x.device))[:, None, None, :]
    n_idx = torch.arange(N, device=x.device)[:, None, None, None]
    c_idx = torch.arange(C, device=x.device)[None, :, None, None]
    return xp[n_idx, c_idx, rows, cols]


def cutout(x, size=16):
    """Zero (= mean) a size x size square at a random centre of each image (may be
clipped at the border)."""
    N, _, H, W = x.shape
    cy = torch.randint(0, H, (N, 1, 1), device=x.device)
    cx = torch.randint(0, W, (N, 1, 1), device=x.device)
    yy = torch.arange(H, device=x.device)[None, :, None]
    xx = torch.arange(W, device=x.device)[None, None, :]
    mask = (yy >= cy - size // 2) & (yy < cy + size // 2) & (xx >= cx - size // 2)
& (xx < cx + size // 2)
    return x * (~mask)[:, None].to(x.dtype)


def mixup(x, y, alpha):
    """Return mixed inputs, both label sets and the mixing coefficient lambda ~
Beta(alpha, alpha)."""
    lam = float(np.random.beta(alpha, alpha))
    p = torch.randperm(len(y), device=x.device)
    return lam * x + (1 - lam) * x[p], y, y[p], lam


def compose(*fs):
    def f(x):
        for g in fs:
            x = g(x)
        return x
    return f


def cosine_warmup(opt, total_steps, warmup_steps=0):
    """Per-iteration LR multiplier: linear warm-up, then cosine decay to 0."""
```

```
    def mult(s):
        if s < warmup_steps:
            return (s + 1) / warmup_steps
        p = (s - warmup_steps) / max(1, total_steps - warmup_steps)
        return 0.5 * (1 + math.cos(math.pi * min(p, 1.0)))
    return torch.optim.lr_scheduler.LambdaLR(opt, mult)


@torch.no_grad()
def evaluate(model, X, y, bs=1000):
    model.eval()
    n = 1000 if QUICK else len(y)
    loss, correct = 0.0, 0
    for i in range(0, n, bs):
        xb, yb = X[i:i + bs].to(device), y[i:i + bs].to(device)
        with torch.autocast(device.type, dtype=torch.float16, enabled=AMP):
            out = model(xb)
        loss += F.cross_entropy(out.float(), yb, reduction="sum").item()
        correct += (out.argmax(1) == yb).sum().item()
    return loss / n, correct / n


def fit(model, opt, n_epochs, X, y, bs=128, aug=None, mixup_alpha=0.0, sched=None,
        label_smoothing=0.0, amp=AMP, eval_train=False, tag=""):
    """Train for n_epochs; returns per-epoch train loss / val loss / val acc (and
clean train acc)."""
    scaler = torch.amp.GradScaler(device.type, enabled=amp)
    H = dict(train_loss=[], val_loss=[], val_acc=[], train_acc=[], iter_loss=[],
time=[])
    for ep in range(n_epochs):
        model.train()
        t0, losses = time.time(), []
        for xb, yb in batches(X, y, bs):
            if aug is not None:
                xb = aug(xb)
            with torch.autocast(device.type, dtype=torch.float16, enabled=amp):
                if mixup_alpha > 0:
                    xb, ya, yb2, lam = mixup(xb, yb, mixup_alpha)
                    out = model(xb)
                    loss = lam * F.cross_entropy(out, ya) + (1 - lam) *
F.cross_entropy(out, yb2)
                else:
                    out = model(xb)
                    loss = F.cross_entropy(out, yb,
label_smoothing=label_smoothing)
            opt.zero_grad(set_to_none=True)
            scaler.scale(loss).backward()
            scaler.step(opt)
            scaler.update()
            if sched is not None:
                sched.step()
            losses.append(loss.detach().float())
        il = torch.stack(losses).cpu().tolist()
        H["iter_loss"] += il
        H["train_loss"].append(float(np.mean(il)))
        vl, va = evaluate(model, X_va, y_va)
```

```
        H["val_loss"].append(vl); H["val_acc"].append(va)
        if eval_train:
            H["train_acc"].append(evaluate(model, X, y)[1])
        H["time"].append(time.time() - t0)
        msg = f"[{tag}] ep {ep + 1:3d}/{n_epochs} loss {H['train_loss'][-1]:.4f}
val_acc {va:.4f}"
        if eval_train:
            msg += f" train_acc {H['train_acc'][-1]:.4f}"
        if ep == 0 or (ep + 1) % max(1, n_epochs // 10) == 0 or ep == n_epochs -
1:
            log(msg + f" ({H['time'][-1]:.1f}s)")
        if not math.isfinite(H["train_loss"][-1]):
            log(f"[{tag}] diverged (non-finite loss), stopping early")
            break
    return H


def n_params(model):
    return sum(p.numel() for p in model.parameters())


def smooth(v, k=50):
    v = np.asarray(v, dtype=float)
    if len(v) < k:
        return v
    return np.convolve(v, np.ones(k) / k, mode="valid")
```

Task 1: Backpropagation

(a) Linear, ReLU and softmax cross-entropy with explicit forward/backward (no autograd)

Each layer caches what it needs in `forward` and implements the chain rule in `backward`:

- **Linear** $Y = XW + b$: $\partial L/\partial W = X^\top G$, $\partial L/\partial b = \sum_n G_n$, $\partial L/\partial X = GW^\top$, where $G = \partial L/\partial Y$.
- **ReLU** $y = \max(0, x)$: $\partial L/\partial x = G \odot \mathbb{1}[x > 0]$.
- **Sigmoid** (needed for 1(d)) $s = \sigma(x)$: $\partial L/\partial x = G \odot s(1 - s)$.
- **Softmax cross-entropy** with mean over the batch: $L = -\frac{1}{N}\sum_n \log p_{n,y_n}$, $\partial L/\partial z = (p - \mathrm{onehot}(y))/N$. The forward pass subtracts the row max (log-sum-exp trick) for numerical stability.

In [4]:
```
class Linear:
    def __init__(self, d_in, d_out, init="he", dtype=torch.float32, dev=device,
gen=None):
        std = math.sqrt(2.0 / d_in) if init == "he" else math.sqrt(2.0 / (d_in +
d_out))  # He or Xavier
        self.W = torch.randn(d_in, d_out, generator=gen, dtype=dtype).to(dev) *
std
```

```
        self.b = torch.zeros(d_out, dtype=dtype, device=dev)
        self.dW = self.db = None

    def forward(self, x):
        self.x = x
        return x @ self.W + self.b

    def backward(self, g):
        self.dW = self.x.T @ g
        self.db = g.sum(0)
        return g @ self.W.T

    def params(self):
        return [(self.W, self.dW), (self.b, self.db)]


class ReLU:
    def forward(self, x):
        self.mask = x > 0
        return x * self.mask

    def backward(self, g):
        return g * self.mask

    def params(self):
        return []


class Sigmoid:
    def forward(self, x):
        self.s = 1.0 / (1.0 + torch.exp(-x))
        return self.s

    def backward(self, g):
        return g * self.s * (1 - self.s)

    def params(self):
        return []


class SoftmaxCrossEntropy:
    def forward(self, z, y):
        z = z - z.max(dim=1, keepdim=True).values
        logp = z - torch.log(torch.exp(z).sum(dim=1, keepdim=True))
        self.p, self.y = torch.exp(logp), y
        return -logp[torch.arange(len(y)), y].mean()

    def backward(self):
        g = self.p.clone()
        g[torch.arange(len(self.y)), self.y] -= 1
        return g / len(self.y)


class MLP:
    def __init__(self, sizes, act="relu", init="he", dtype=torch.float32,
dev=device, seed=SEED):
```

```
        gen = torch.Generator().manual_seed(seed)
        Act = ReLU if act == "relu" else Sigmoid
        self.layers = []
        for i, (a, b) in enumerate(zip(sizes[:-1], sizes[1:])):
            self.layers.append(Linear(a, b, init, dtype, dev, gen))
            if i < len(sizes) - 2:
                self.layers.append(Act())
        self.loss_fn = SoftmaxCrossEntropy()

    def forward(self, x):
        for layer in self.layers:
            x = layer.forward(x)
        return x

    def loss(self, x, y):
        return self.loss_fn.forward(self.forward(x), y)

    def backward(self):
        g = self.loss_fn.backward()
        for layer in reversed(self.layers):
            g = layer.backward(g)
        return g  # dL/dx

    def linears(self):
        return [l for l in self.layers if isinstance(l, Linear)]

    def sgd_step(self, lr):
        for lin in self.linears():
            lin.W -= lr * lin.dW
            lin.b -= lr * lin.db
```

(b) Gradient check against autograd and centred finite differences

A small MLP (12-10-8-5) in float64 on a random batch. For every parameter tensor we compare the manual gradient g_m with (i) the autograd gradient of an equivalent `torch` graph sharing the same weights, and (ii) centred finite differences $\big(L(\theta + he_i) - L(\theta - he_i)\big)/2h$ with $h = 10^{-6}$. The relative error is $\|g_m - g\|_2 / \max(\|g_m\|_2 + \|g\|_2, 10^{-12})$. The check is done for both the ReLU and the sigmoid network (the latter is used in 1(d)).

In [5]:
```
def rel_err(a, b):
    return ((a - b).norm() / max((a.norm() + b.norm()).item(), 1e-12)).item()


def grad_check(act):
    torch.manual_seed(1)
    net = MLP([12, 10, 8, 5], act=act, dtype=torch.float64, dev="cpu", seed=1)
    for lin in net.linears():  # non-zero biases so that their gradients are
tested properly
        lin.b += 0.1 * torch.randn_like(lin.b)
    x = torch.randn(6, 12, dtype=torch.float64)
    y = torch.randint(0, 5, (6,))
```

```
    # manual gradients
    net.loss(x, y)
    dx_manual = net.backward()
    manual = {}
    for k, lin in enumerate(net.linears()):
        manual[f"W{k + 1}"], manual[f"b{k + 1}"] = lin.dW.clone(), lin.db.clone()
    manual["x"] = dx_manual

    # autograd on the same weights
    leaves = {}
    h = x.clone().requires_grad_(True)
    leaves["x"] = h
    lins = net.linears()
    for k, lin in enumerate(lins):
        W = lin.W.clone().requires_grad_(True); b =
lin.b.clone().requires_grad_(True)
        leaves[f"W{k + 1}"], leaves[f"b{k + 1}"] = W, b
        h = h @ W + b
        if k < len(lins) - 1:
            h = torch.relu(h) if act == "relu" else torch.sigmoid(h)
    F.cross_entropy(h, y).backward()

    # centred finite differences
    def fd(tensor, eps=1e-6):
        g = torch.zeros_like(tensor)
        flat, gflat = tensor.view(-1), g.view(-1)
        for i in range(flat.numel()):
            old = flat[i].item()
            flat[i] = old + eps; lp = net.loss(x, y).item()
            flat[i] = old - eps; lm = net.loss(x, y).item()
            flat[i] = old
            gflat[i] = (lp - lm) / (2 * eps)
        return g

    rows = []
    for name, g_m in manual.items():
        if name == "x":
            target = x
        else:
            k = int(name[1:]) - 1
            target = lins[k].W if name[0] == "W" else lins[k].b
        g_auto = leaves[name].grad
        g_fd = fd(target)
        rows.append((act, name, tuple(g_m.shape), rel_err(g_m, g_auto),
rel_err(g_m, g_fd),
                     torch.allclose(g_m, g_auto), torch.allclose(g_m, g_fd,
atol=1e-7)))
    return rows


gc = pd.DataFrame([r for act in ("relu", "sigmoid") for r in grad_check(act)],
                  columns=["activation", "gradient", "shape", "rel. error vs
autograd", "rel. error vs FD",
                           "allclose (autograd)", "allclose (FD)"])
display(gc.style.format({"rel. error vs autograd": "{:.2e}", "rel. error vs FD": "
```

```
{:.2e}"}).hide(axis="index"))
print("all gradients match:", bool(gc["allclose (autograd)"].all() and
gc["allclose (FD)"].all()))
```

activation	gradient	shape	rel. error vs autograd	rel. error vs FD	allclose (autograd)	allclose (FD)
relu	W1	(12, 10)	7.31e-17	6.95e-10	True	True
relu	b1	(10,)	5.76e-17	5.73e-10	True	True
relu	W2	(10, 8)	7.16e-17	4.20e-10	True	True
relu	b2	(8,)	1.03e-16	6.51e-10	True	True
relu	W3	(8, 5)	3.98e-17	3.09e-10	True	True
relu	b3	(5,)	1.61e-17	3.77e-10	True	True
relu	x	(6, 12)	5.83e-17	6.39e-10	True	True
sigmoid	W1	(12, 10)	1.35e-16	7.65e-09	True	True
sigmoid	b1	(10,)	1.62e-16	1.24e-08	True	True
sigmoid	W2	(10, 8)	1.09e-16	3.10e-09	True	True
sigmoid	b2	(8,)	9.99e-17	1.13e-09	True	True
sigmoid	W3	(8, 5)	3.04e-17	5.72e-10	True	True
sigmoid	b3	(5,)	3.54e-17	4.01e-10	True	True
sigmoid	x	(6, 12)	1.28e-16	7.63e-09	True	True

```
all gradients match: True
```

Interpretation. For every parameter tensor (and for the input gradient) the manual backward pass agrees with autograd to a relative error of about 10^{-16}, i.e. machine precision in float64: the two compute the same expression, only the order of floating-point operations differs. Against centred finite differences the relative error is 10^{-10}–10^{-8}, which is exactly what we expect from the method: the truncation error is $O(h^2) \approx 10^{-12}$ and the round-off error is $O(\epsilon_{mach}/h) \approx 10^{-10}$. `torch.allclose` returns `True` in all cases, for both the ReLU and the sigmoid network, so the hand-written Linear / ReLU / Sigmoid / softmax cross-entropy backward passes are correct. (ReLU is not differentiable at 0, but with random real-valued inputs no pre-activation lands within h of 0, so finite differences are valid.)

(c) Training the MLP with SGD on flattened CIFAR-10

MLP 3072 → 512 → 256 → 10 with ReLU, He initialization, mini-batch SGD (batch 128, learning rate 0.05 with cosine decay), 30 epochs, using only the hand-written forward/backward above (no autograd).

In [6]:
```
def train_manual_mlp(n_epochs=30, lr0=0.05, bs=128):
    set_seed()
    net = MLP([3072, 512, 256, 10], act="relu", init="he")
    Xf_tr, Xf_va = X_tr.reshape(len(X_tr), -1), X_va.reshape(len(X_va), -1)
    steps = n_epochs * math.ceil(len(y_tr) / bs)
    H = dict(iter_loss=[], train_loss=[], val_loss=[], val_acc=[], train_acc=[])
```

```
    step = 0
    for ep in range(n_epochs):
        losses = []
        for xb, yb in batches(Xf_tr, y_tr, bs):
            lr = lr0 * 0.5 * (1 + math.cos(math.pi * step / steps))
            losses.append(net.loss(xb, yb))
            net.backward()
            net.sgd_step(lr)
            step += 1
        il = torch.stack(losses).cpu().tolist()
        H["iter_loss"] += il
        H["train_loss"].append(float(np.mean(il)))

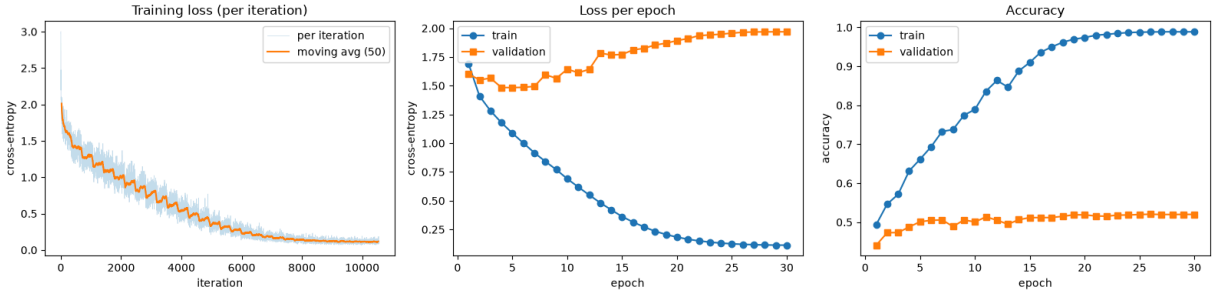
        def acc_loss(Xf, y):
            n = 1000 if QUICK else len(y)
            out = torch.cat([net.forward(Xf[i:i + 1000]) for i in range(0, n,
1000)])
            return net.loss_fn.forward(out, y[:n]).item(), (out.argmax(1) ==
y[:n]).float().mean().item()

        vl, va = acc_loss(Xf_va, y_va)
        H["val_loss"].append(vl); H["val_acc"].append(va)
        H["train_acc"].append(acc_loss(Xf_tr, y_tr)[1])
        log(f"ep {ep + 1:2d} train_loss {H['train_loss'][-1]:.4f} train_acc
{H['train_acc'][-1]:.4f} "
            f"val_loss {vl:.4f} val_acc {va:.4f}")
    return H


H1c = cached("task1c_mlp", lambda: train_manual_mlp(epochs(30)))

fig, ax = plt.subplots(1, 3, figsize=(16, 4))
ax[0].plot(H1c["iter_loss"], alpha=0.25, lw=0.5, label="per iteration")
ax[0].plot(np.arange(len(smooth(H1c["iter_loss"]))) + 25,
smooth(H1c["iter_loss"]), label="moving avg (50)")
ax[0].set(xlabel="iteration", ylabel="cross-entropy", title="Training loss (per
iteration)"); ax[0].legend()
ep = np.arange(1, len(H1c["train_loss"]) + 1)
ax[1].plot(ep, H1c["train_loss"], "o-", label="train"); ax[1].plot(ep,
H1c["val_loss"], "s-", label="validation")
ax[1].set(xlabel="epoch", ylabel="cross-entropy", title="Loss per epoch");
ax[1].legend()
ax[2].plot(ep, H1c["train_acc"], "o-", label="train"); ax[2].plot(ep,
H1c["val_acc"], "s-", label="validation")
ax[2].set(xlabel="epoch", ylabel="accuracy", title="Accuracy"); ax[2].legend()
plt.tight_layout(); savefig("task1c_mlp"); plt.show()
print(f"Final validation accuracy: {H1c['val_acc'][-1]:.4f} (best
{max(H1c['val_acc']):.4f})")
```

```
[task1c_mlp: loaded saved run; its training log was:]
  ep  1 train_loss 1.6921 train_acc 0.4939 val_loss 1.6016 val_acc 0.4410
  ep  2 train_loss 1.4097 train_acc 0.5465 val_loss 1.5511 val_acc 0.4730
  ep  3 train_loss 1.2818 train_acc 0.5716 val_loss 1.5657 val_acc 0.4734
  ep  4 train_loss 1.1786 train_acc 0.6315 val_loss 1.4843 val_acc 0.4878
  ep  5 train_loss 1.0869 train_acc 0.6616 val_loss 1.4841 val_acc 0.5012
  ep  6 train_loss 1.0011 train_acc 0.6935 val_loss 1.4861 val_acc 0.5048
  ep  7 train_loss 0.9169 train_acc 0.7329 val_loss 1.4939 val_acc 0.5058
  ep  8 train_loss 0.8416 train_acc 0.7376 val_loss 1.5961 val_acc 0.4890
  ep  9 train_loss 0.7701 train_acc 0.7745 val_loss 1.5643 val_acc 0.5058
  ep 10 train_loss 0.6905 train_acc 0.7895 val_loss 1.6438 val_acc 0.5006
  ep 11 train_loss 0.6193 train_acc 0.8361 val_loss 1.6131 val_acc 0.5136
  ep 12 train_loss 0.5499 train_acc 0.8638 val_loss 1.6423 val_acc 0.5052
  ep 13 train_loss 0.4822 train_acc 0.8467 val_loss 1.7841 val_acc 0.4948
  ep 14 train_loss 0.4214 train_acc 0.8883 val_loss 1.7671 val_acc 0.5064
  ep 15 train_loss 0.3618 train_acc 0.9102 val_loss 1.7720 val_acc 0.5118
  ep 16 train_loss 0.3144 train_acc 0.9363 val_loss 1.8113 val_acc 0.5118
  ep 17 train_loss 0.2720 train_acc 0.9500 val_loss 1.8249 val_acc 0.5120
  ep 18 train_loss 0.2347 train_acc 0.9618 val_loss 1.8541 val_acc 0.5150
  ep 19 train_loss 0.2065 train_acc 0.9696 val_loss 1.8695 val_acc 0.5190
  ep 20 train_loss 0.1836 train_acc 0.9738 val_loss 1.8924 val_acc 0.5188
  ep 21 train_loss 0.1651 train_acc 0.9798 val_loss 1.9101 val_acc 0.5160
  ep 22 train_loss 0.1510 train_acc 0.9820 val_loss 1.9350 val_acc 0.5154
  ep 23 train_loss 0.1400 train_acc 0.9847 val_loss 1.9417 val_acc 0.5182
  ep 24 train_loss 0.1316 train_acc 0.9867 val_loss 1.9480 val_acc 0.5184
  ep 25 train_loss 0.1257 train_acc 0.9877 val_loss 1.9570 val_acc 0.5198
  ep 26 train_loss 0.1210 train_acc 0.9882 val_loss 1.9646 val_acc 0.5204
  ep 27 train_loss 0.1178 train_acc 0.9886 val_loss 1.9673 val_acc 0.5198
  ep 28 train_loss 0.1155 train_acc 0.9887 val_loss 1.9682 val_acc 0.5198
  ep 29 train_loss 0.1142 train_acc 0.9885 val_loss 1.9687 val_acc 0.5200
  ep 30 train_loss 0.1136 train_acc 0.9886 val_loss 1.9689 val_acc 0.5200
```




```
Final validation accuracy: 0.5200 (best 0.5204)
```

Interpretation. The hand-written MLP trains correctly with SGD: the training loss falls smoothly from 2.3 to about 0.11 (the small steps in the per-iteration curve are epoch boundaries, where the reshuffled data starts again) and the final **validation accuracy is 52.0%** (best 52.0%), far above the 10% chance level and typical for an MLP on raw CIFAR-10 pixels. However, the model overfits heavily: train accuracy reaches 98.9%, while the validation loss is lowest at epoch 5 (1.48) and then *increases* to 1.97 although the validation accuracy stays flat around 51–52%: the network becomes over-confident on the validation images it gets wrong. An MLP on flattened pixels has no notion of locality or translation invariance (1.7M parameters, each pixel treated as an independent feature), so it memorises the training set instead of learning features that generalise; this is the motivation for the convolutional networks in the following tasks (which reach >70% after only a few epochs).

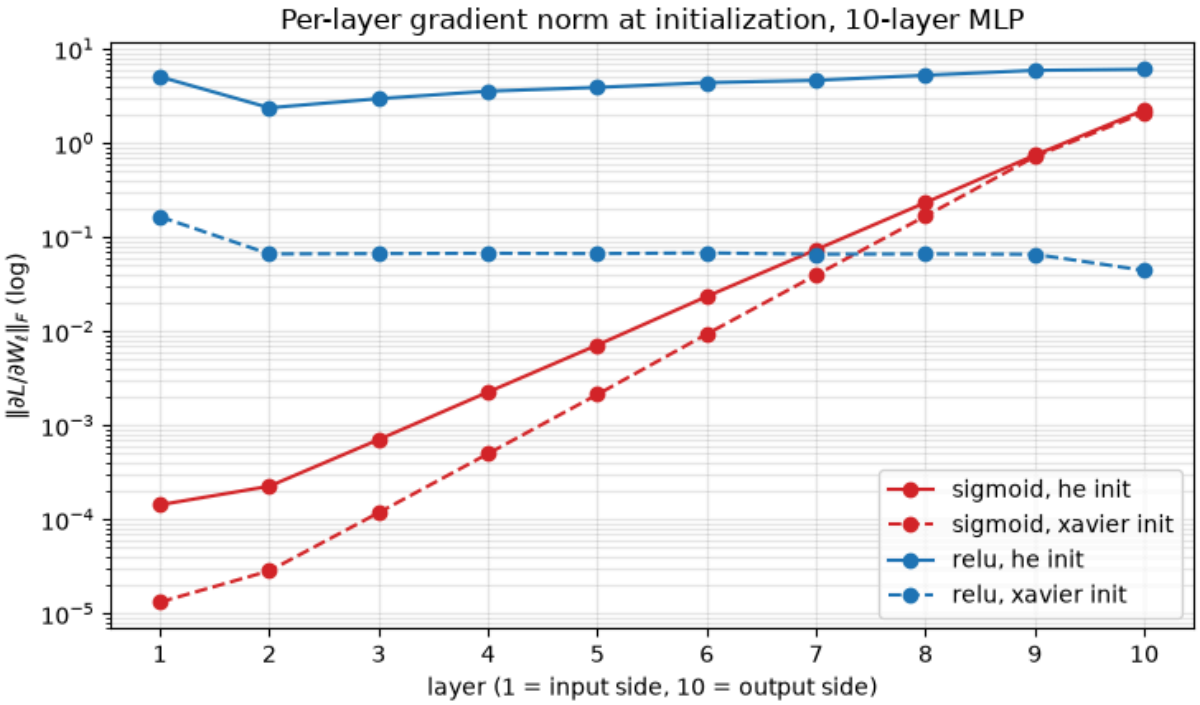
(d) Per-layer gradient norms at initialization in a 10-layer MLP: sigmoid vs ReLU

A 10-layer MLP (3072 → 9 × 256 hidden → 10) is evaluated at initialization on a batch of 512 real training images, and $\|\partial L/\partial W_\ell\|_F$ is recorded for every layer ℓ (layer 1 is next to the input). We use the same network with the activation as the only change, for two standard initializations (He: $\mathrm{Var}(W) = 2/d_{in}$; Xavier: $\mathrm{Var}(W) = 2/(d_{in} + d_{out})$), averaged over 5 random initializations.

In [7]:
```
def layer_grad_norms(act, init, seeds=range(5)):
    xb, yb = X_tr[:512].reshape(512, -1), y_tr[:512]
    norms = []
    for s in seeds:
        net = MLP([3072] + [256] * 9 + [10], act=act, init=init, seed=s)
        net.loss(xb, yb)
        net.backward()
        norms.append([lin.dW.norm().item() for lin in net.linears()])
    return np.mean(norms, axis=0)


fig, ax = plt.subplots(figsize=(7.5, 4.5))
gn = {}
for act, c in (("sigmoid", "tab:red"), ("relu", "tab:blue")):
    for init, ls in (("he", "-"), ("xavier", "--")):
        g = gn[f"{act}, {init} init"] = layer_grad_norms(act, init)
        ax.semilogy(range(1, 11), g, ls, color=c, marker="o", label=f"{act},
{init} init")
ax.set(xlabel="layer (1 = input side, 10 = output side)", ylabel=r"$\|\partial
L/\partial W_\ell\|_F$ (log)",
       title="Per-layer gradient norm at initialization, 10-layer MLP")
ax.set_xticks(range(1, 11)); ax.grid(True, which="both", alpha=0.3); ax.legend()
plt.tight_layout(); savefig("task1d_gradnorms"); plt.show()

gn = pd.DataFrame(gn, index=[f"layer {i}" for i in range(1, 11)]).T
gn["layer 1 / layer 10"] = gn["layer 1"] / gn["layer 10"]
display(gn.style.format("{:.1e}"))
```



	layer 1	layer 2	layer 3	layer 4	layer 5	layer 6	layer 7	layer 8	layer 9	layer 10	layer 1 / layer 10
sigmoid, he init	1.4e-04	2.3e-04	7.1e-04	2.3e-03	7.1e-03	2.3e-02	7.4e-02	2.3e-01	7.4e-01	2.2e+00	6.4e-05
sigmoid, xavier init	1.3e-05	2.8e-05	1.2e-04	5.0e-04	2.1e-03	9.3e-03	4.0e-02	1.7e-01	7.2e-01	2.1e+00	6.3e-06
relu, he init	5.0e+00	2.4e+00	3.0e+00	3.5e+00	3.9e+00	4.4e+00	4.6e+00	5.2e+00	5.9e+00	6.1e+00	8.3e-01
relu, xavier init	1.6e-01	6.6e-02	6.7e-02	6.7e-02	6.7e-02	6.8e-02	6.6e-02	6.6e-02	6.6e-02	4.4e-02	3.7e+00

Explanation. Backpropagation multiplies the incoming gradient, layer by layer, by $W_\ell^\top$ and by the derivative of the activation: $\delta_\ell = (W_{\ell+1}^\top \delta_{\ell+1}) \odot \phi'(z_\ell)$, and $\partial L/\partial W_\ell = h_{\ell-1}^\top \delta_\ell$.

- **Sigmoid:** $\sigma'(z) = \sigma(z)(1-\sigma(z)) \le 0.25$, so every layer shrinks the gradient by a factor of roughly 3–5 (the weights cannot compensate). Over 9 activations the gradient norm decays geometrically, which is a straight line on the log plot: the first layer receives a gradient about 10^4 (He init) to 10^5 (Xavier) times smaller than the last layer. The early layers would therefore barely learn with SGD: this is the **vanishing-gradient problem**. Sigmoid outputs are also not zero-centred, which makes it worse.
- **ReLU:** $\phi'(z) \in \{0, 1\}$ and, for symmetric pre-activations, half of the units are active, which halves the variance of the back-propagated signal. He initialization ($\mathrm{Var}(W) = 2/d_{in}$) compensates exactly for this factor $1/2$, so the gradient norm stays roughly constant across all ten layers (the flat solid blue line). With Xavier initialization and ReLU the hidden weights have variance $1/d_{in}$ (half of He's), so both the forward

activations h_ℓ (which shrink with depth) and the back-propagated δ_ℓ (which shrink towards the input) lose a factor $\sqrt{2}$ per layer. In $\partial L/\partial W_\ell = h_{\ell-1}^\top \delta_\ell$ the two effects cancel, so the weight-gradient norm is flat but about 100× smaller than with He init, i.e. the whole network learns slowly and the signal itself (activations and δ) still vanishes exponentially with depth.

The first-layer value for ReLU is slightly higher simply because W_1 has 3072×256 entries (the Frobenius norm grows with the number of entries). Conclusion: ReLU together with variance-preserving (He) initialization keeps gradients well-scaled at initialization, which is why deep networks use it (and batch norm / residual connections, Task 3, to keep it that way during training).

Task 2: Optimizers

(a) SGD with momentum and Adam (with bias correction) as `torch.optim.Optimizer` subclasses

- **SGD + momentum** (PyTorch convention, no dampening): $g \leftarrow \nabla_\theta L + \lambda\theta$, $b_t = \mu b_{t-1} + g$ (with $b_1 = g$), $\theta \leftarrow \theta - \eta\, b_t$.
- **Adam**: $m_t = \beta_1 m_{t-1} + (1-\beta_1) g$, $v_t = \beta_2 v_{t-1} + (1-\beta_2) g^2$, bias correction $\hat{m}_t = m_t/(1-\beta_1^t)$, $\hat{v}_t = v_t/(1-\beta_2^t)$, $\theta \leftarrow \theta - \eta\, \hat{m}_t/(\sqrt{\hat{v}_t} + \epsilon)$.

In [8]:
```
class MySGD(torch.optim.Optimizer):
    def __init__(self, params, lr, momentum=0.0, weight_decay=0.0):
        super().__init__(params, dict(lr=lr, momentum=momentum,
weight_decay=weight_decay))

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, mu, wd = group["lr"], group["momentum"], group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad if wd == 0 else p.grad + wd * p
                if mu != 0:
                    state = self.state[p]
                    if "momentum_buffer" not in state:
                        state["momentum_buffer"] = g.clone()
                    else:
                        state["momentum_buffer"].mul_(mu).add_(g)
                    g = state["momentum_buffer"]
                p.add_(g, alpha=-lr)
```

```
        return loss


class MyAdam(torch.optim.Optimizer):
    def __init__(self, params, lr=1e-3, betas=(0.9, 0.999), eps=1e-8,
weight_decay=0.0):
        super().__init__(params, dict(lr=lr, betas=betas, eps=eps,
weight_decay=weight_decay))

    @torch.no_grad()
    def step(self, closure=None):
        loss = None
        if closure is not None:
            with torch.enable_grad():
                loss = closure()
        for group in self.param_groups:
            lr, (b1, b2), eps, wd = group["lr"], group["betas"], group["eps"],
group["weight_decay"]
            for p in group["params"]:
                if p.grad is None:
                    continue
                g = p.grad if wd == 0 else p.grad + wd * p
                state = self.state[p]
                if not state:
                    state["t"] = 0
                    state["m"] = torch.zeros_like(p)
                    state["v"] = torch.zeros_like(p)
                state["t"] += 1
                t, m, v = state["t"], state["m"], state["v"]
                m.mul_(b1).add_(g, alpha=1 - b1)
                v.mul_(b2).addcmul_(g, g, value=1 - b2)
                m_hat = m / (1 - b1 ** t)
                v_hat = v / (1 - b2 ** t)
                p.addcdiv_(m_hat, v_hat.sqrt().add_(eps), value=-lr)
        return loss
```

Equivalence test. Two copies of the same small CNN (float64) are trained for 50 steps on the same CIFAR-10 mini-batches, one with our optimizer and one with the PyTorch reference; all parameters are compared at the end with `torch.allclose` (and the max absolute difference is reported, next to how far the parameters moved, to show that the comparison is not trivial).

In [9]:
```
def compare_optimizers(make_mine, make_ref, steps=50):
    torch.manual_seed(0)
    net = nn.Sequential(nn.Conv2d(3, 8, 3, padding=1), nn.ReLU(), nn.MaxPool2d(2),
                        nn.Conv2d(8, 16, 3, padding=1), nn.ReLU(),
nn.AdaptiveAvgPool2d(1),
                        nn.Flatten(), nn.Linear(16, 10)).double().to(device)
    a, b = copy.deepcopy(net), copy.deepcopy(net)
    oa, ob = make_mine(a.parameters()), make_ref(b.parameters())
    for s in range(steps):
        xb = X_tr[s * 64:(s + 1) * 64].double(); yb = y_tr[s * 64:(s + 1) * 64]
        for model, opt in ((a, oa), (b, ob)):
            opt.zero_grad()
```

```
            F.cross_entropy(model(xb), yb).backward()
            opt.step()
    diff = max((pa - pb).abs().max().item() for pa, pb in zip(a.parameters(),
b.parameters()))
    same = all(torch.allclose(pa, pb) for pa, pb in zip(a.parameters(),
b.parameters()))
    moved = max((pa - p0).abs().max().item() for pa, p0 in zip(a.parameters(),
net.parameters()))
    return same, diff, moved


tests = {
    "SGD (no momentum)": (lambda p: MySGD(p, lr=0.05), lambda p:
torch.optim.SGD(p, lr=0.05)),
    "SGD momentum 0.9": (lambda p: MySGD(p, lr=0.05, momentum=0.9),
                         lambda p: torch.optim.SGD(p, lr=0.05, momentum=0.9)),
    "SGD momentum 0.9 + wd 5e-4": (lambda p: MySGD(p, lr=0.05, momentum=0.9,
weight_decay=5e-4),
                                   lambda p: torch.optim.SGD(p, lr=0.05,
momentum=0.9, weight_decay=5e-4)),
    "Adam": (lambda p: MyAdam(p, lr=1e-3), lambda p: torch.optim.Adam(p, lr=1e-
3)),
    "Adam betas (0.8, 0.99) + wd 1e-3": (lambda p: MyAdam(p, lr=3e-3, betas=(0.8,
0.99), weight_decay=1e-3),
                                         lambda p: torch.optim.Adam(p, lr=3e-3,
betas=(0.8, 0.99),

weight_decay=1e-3)),
}
opt_check = pd.DataFrame([(name, *compare_optimizers(mine, ref)) for name, (mine,
ref) in tests.items()],
                         columns=["optimizer (ours vs torch.optim)", "allclose",
"max |difference|",
                                  "max |parameter change|"])
display(opt_check.style.format({"max |difference|": "{:.2e}", "max |parameter
change|": "{:.2e}"})
        .hide(axis="index"))
```

optimizer (ours vs torch.optim)	allclose	max |difference|	max |parameter change|
SGD (no momentum)	True	0.00e+00	6.72e-02
SGD momentum 0.9	True	0.00e+00	2.74e-01
SGD momentum 0.9 + wd 5e-4	True	5.55e-17	2.73e-01
Adam	True	8.33e-17	5.32e-02
Adam betas (0.8, 0.99) + wd 1e-3	True	8.33e-17	1.77e-01

Interpretation. After 50 steps on real CIFAR-10 batches, the parameters produced by `MySGD` and `MyAdam` are identical to those of `torch.optim.SGD` / `torch.optim.Adam` (`allclose = True`), with a maximum absolute difference of 0 for SGD and $\sim 10^{-16}$ (float64 round-off) for the variants with weight decay and for Adam, while the parameters themselves moved by 5×10^{-2}–3×10^{-1}, so the test is not trivial. The tiny Adam differences come only from the order of operations (PyTorch folds the bias corrections into

the step size and the denominator, $\theta \leftarrow \theta - \frac{\eta}{1-\beta_1^t} m_t / (\sqrt{v_t}/\sqrt{1-\beta_2^t} + \epsilon)$, which is algebraically the same as our $\hat{m}_t/(\sqrt{\hat{v}_t}+\epsilon)$). The momentum buffer convention ($b_1 = g_1$, no dampening) and the L2-style weight decay added to the gradient also match PyTorch.

(b) Trajectories on $f(x, y) = \frac{1}{2}(x^2 + 50y^2)$

$\nabla f = (x,\ 50y)$, so the Hessian is $\mathrm{diag}(1, 50)$ (condition number 50). Plain gradient descent is stable only for $\eta < 2/50 = 0.04$. All three methods start at $(-9, 1.5)$, run for 100 steps and use our own optimizer classes from (a); the gradient is computed by autograd.

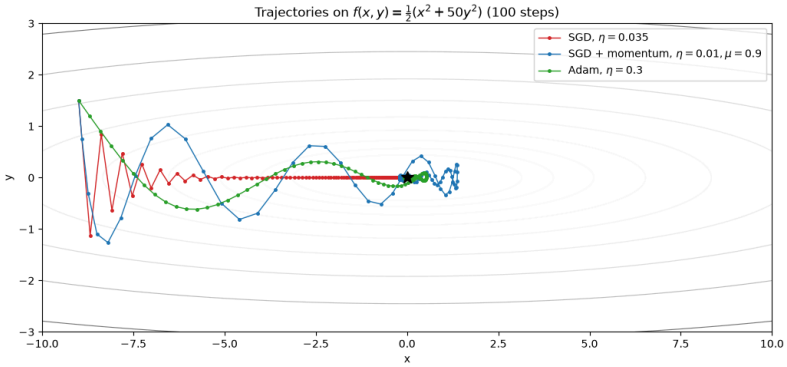
In [10]:
```
def f_quad(p):
    return 0.5 * (p[0] ** 2 + 50 * p[1] ** 2)

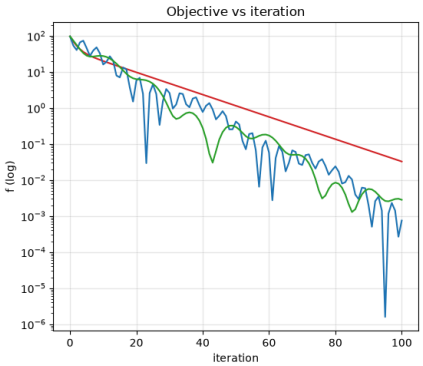

def trajectory(make_opt, start=(-9.0, 1.5), steps=100):
    p = torch.tensor(start, dtype=torch.float64, requires_grad=True)
    opt = make_opt([p])
    path = [p.detach().clone().numpy()]
    for _ in range(steps):
        opt.zero_grad()
        f_quad(p).backward()
        opt.step()
        path.append(p.detach().clone().numpy())
    return np.array(path)


runs_2b = {
    r"SGD, $\eta=0.035$": (lambda p: MySGD(p, lr=0.035), "tab:red"),
    r"SGD + momentum, $\eta=0.01, \mu=0.9$": (lambda p: MySGD(p, lr=0.01,
momentum=0.9), "tab:blue"),
    r"Adam, $\eta=0.3$": (lambda p: MyAdam(p, lr=0.3), "tab:green"),
}
paths = {k: trajectory(v[0]) for k, v in runs_2b.items()}

xs, ys = np.meshgrid(np.linspace(-10, 10, 400), np.linspace(-3, 3, 300))
fig, ax = plt.subplots(1, 2, figsize=(16, 5), gridspec_kw=dict(width_ratios=[2,
1]))
ax[0].contour(xs, ys, 0.5 * (xs ** 2 + 50 * ys ** 2), levels=np.logspace(-1, 2.6,
18), cmap="Greys",
              linewidths=0.8)
for k, path in paths.items():
    ax[0].plot(path[:, 0], path[:, 1], "o-", ms=2.5, lw=1, color=runs_2b[k][1],
label=k)
    ax[1].semilogy([f_quad(q) for q in path], color=runs_2b[k][1], label=k)
ax[0].plot(0, 0, "k*", ms=12)
ax[0].set(xlabel="x", ylabel="y", title=r"Trajectories on $f(x,y)=\frac{1}{2}
(x^2+50y^2)$ (100 steps)",
          xlim=(-10, 10), ylim=(-3, 3))
ax[0].legend(loc="upper right")
ax[1].set(xlabel="iteration", ylabel="f (log)", title="Objective vs iteration");
ax[1].grid(alpha=0.3)
plt.tight_layout(); savefig("task2b_trajectories"); plt.show()
```

```
display(pd.DataFrame([(k.replace("$", "").replace("\\eta", "lr").replace("\\mu",
"momentum"), *path[-1],
                       f_quad(path[-1])) for k, path in paths.items()],
                      columns=["method", "x after 100 steps", "y after 100 steps",
"f after 100 steps"])
        .style.format({"x after 100 steps": "{:.4f}", "y after 100 steps": "
{:.4f}", "f after 100 steps": "{:.2e}"})
        .hide(axis="index"))
```





method	x after 100 steps	y after 100 steps	f after 100 steps
SGD, lr=0.035	-0.2553	0.0000	3.26e-02
SGD + momentum, lr=0.01, momentum=0.9	0.0237	0.0043	7.51e-04
Adam, lr=0.3	-0.0580	-0.0067	2.82e-03

Explanation. Gradient descent on this quadratic acts on each coordinate separately: $x_{t+1} = (1 - \eta)x_t$ and $y_{t+1} = (1 - 50\eta)y_t$.

- **SGD** ($\eta = 0.035$, close to the stability limit $2/50$): along the steep y direction the factor is $1 - 50\eta = -0.75$, so y flips sign every step (zig-zag across the valley) while it decays; along the shallow x direction the factor is 0.965, so progress is very slow. The learning rate is limited by the *largest* curvature, while the convergence speed is set by the *smallest* one: after 100 steps it is still at $x = -0.26$ ($f = 3 \times 10^{-2}$). This is the classic ill-conditioning problem (condition number 50).
- **SGD + momentum** ($\eta = 0.01$, $\mu = 0.9$): the velocity accumulates the consistent x-component of the gradient, so the effective step along the valley is $\eta/(1 - \mu) = 0.1$, ten times larger, while the alternating y-components largely cancel. The trajectory is under-damped (it overshoots the minimum and oscillates in both coordinates, the "heavy ball" behaviour), but the per-step contraction is $\sqrt{\mu} \approx 0.95$ in *both* directions instead of 0.965 in x, so it reaches the lowest objective ($f \approx 7 \times 10^{-4}$).
- **Adam** ($\eta = 0.3$): the update $\hat{m}/\sqrt{\hat{v}}$ divides each coordinate by its own gradient magnitude, so the first steps have size $\approx \eta$ in both x and y (the path starts along the diagonal regardless of the 50× different curvatures). This per-parameter scaling removes most of the ill-conditioning: y is driven to 0 quickly and x moves at a roughly constant speed, giving a smooth curved path. Near the minimum the normalised step does not shrink with the gradient, so Adam oscillates in a small region ($f \approx 3 \times 10^{-3}$) unless the learning rate is decayed.

(c) A small CNN trained with each optimizer at three learning rates

Small CNN without batch norm (so that the optimizer, not normalization, determines stability): `[conv3-32, conv3-32, pool] → [conv3-64, conv3-64, pool] → [conv3-128, pool] → GAP → FC-10`, ReLU, about 0.14M parameters. Each of the 9 runs uses the same initialization (seed), the same data order, batch size 128, a constant learning rate from the common grid $\{10^{-3}, 10^{-2}, 10^{-1}\}$, 10 epochs, no augmentation, full fp32 precision, and our optimizer implementations from (a) (momentum 0.9; Adam with default betas).

In [11]:
```
class SmallCNN(nn.Module):
    def __init__(self, bn=False, p_drop=0.0, widths=(32, 64, 128)):
        super().__init__()
        c1, c2, c3 = widths

        def conv(cin, cout):
            layers = [nn.Conv2d(cin, cout, 3, padding=1, bias=not bn)]
            if bn:
                layers.append(nn.BatchNorm2d(cout))
            return layers + [nn.ReLU(inplace=True)]

        self.features = nn.Sequential(
            *conv(3, c1), *conv(c1, c1), nn.MaxPool2d(2), nn.Dropout(p_drop),
            *conv(c1, c2), *conv(c2, c2), nn.MaxPool2d(2), nn.Dropout(p_drop),
            *conv(c2, c3), nn.MaxPool2d(2))
        self.head = nn.Sequential(nn.AdaptiveAvgPool2d(1), nn.Flatten(),
nn.Dropout(p_drop), nn.Linear(c3, 10))

    def forward(self, x):
        return self.head(self.features(x))


print("SmallCNN parameters:", n_params(SmallCNN()))

OPTS_2C = {
    "SGD": lambda p, lr: MySGD(p, lr=lr),
    "SGD+momentum": lambda p, lr: MySGD(p, lr=lr, momentum=0.9),
    "Adam": lambda p, lr: MyAdam(p, lr=lr),
}
LRS_2C = [1e-3, 1e-2, 1e-1]


def run_2c(opt_name, lr):
    set_seed()
    model = SmallCNN().to(device)
    opt = OPTS_2C[opt_name](model.parameters(), lr)
    return fit(model, opt, epochs(10), X_tr, y_tr, bs=128, amp=False, tag=f"
{opt_name} lr={lr:g}")


R2c = {(o, lr): cached(f"task2c_{o}_lr{lr:g}", lambda o=o, lr=lr: run_2c(o, lr))
       for o in OPTS_2C for lr in LRS_2C}
```

```
SmallCNN parameters: 140714
[task2c_SGD_lr0.001: loaded saved run; its training log was:]
  [SGD lr=0.001] ep   1/10 loss 2.3037 val_acc 0.1022 (8.7s)
  [SGD lr=0.001] ep   2/10 loss 2.3035 val_acc 0.1030 (5.0s)
  [SGD lr=0.001] ep   3/10 loss 2.3033 val_acc 0.1032 (5.0s)
  [SGD lr=0.001] ep   4/10 loss 2.3031 val_acc 0.1028 (5.0s)
  [SGD lr=0.001] ep   5/10 loss 2.3029 val_acc 0.1030 (5.0s)
  [SGD lr=0.001] ep   6/10 loss 2.3027 val_acc 0.1044 (5.0s)
  [SGD lr=0.001] ep   7/10 loss 2.3025 val_acc 0.1032 (5.0s)
  [SGD lr=0.001] ep   8/10 loss 2.3023 val_acc 0.1054 (5.0s)
  [SGD lr=0.001] ep   9/10 loss 2.3021 val_acc 0.1056 (5.0s)
  [SGD lr=0.001] ep  10/10 loss 2.3019 val_acc 0.1058 (5.0s)
[task2c_SGD_lr0.01: loaded saved run; its training log was:]
  [SGD lr=0.01] ep   1/10 loss 2.3029 val_acc 0.1058 (5.0s)
  [SGD lr=0.01] ep   2/10 loss 2.3008 val_acc 0.1370 (5.0s)
  [SGD lr=0.01] ep   3/10 loss 2.2963 val_acc 0.1206 (5.0s)
  [SGD lr=0.01] ep   4/10 loss 2.2750 val_acc 0.1268 (5.0s)
  [SGD lr=0.01] ep   5/10 loss 2.2139 val_acc 0.1756 (5.0s)
  [SGD lr=0.01] ep   6/10 loss 2.1316 val_acc 0.2198 (5.0s)
  [SGD lr=0.01] ep   7/10 loss 2.0185 val_acc 0.2792 (5.0s)
  [SGD lr=0.01] ep   8/10 loss 1.9192 val_acc 0.2996 (5.0s)
  [SGD lr=0.01] ep   9/10 loss 1.8372 val_acc 0.3244 (5.0s)
  [SGD lr=0.01] ep  10/10 loss 1.7819 val_acc 0.3440 (5.0s)
[task2c_SGD_lr0.1: loaded saved run; its training log was:]
  [SGD lr=0.1] ep   1/10 loss 2.1774 val_acc 0.2356 (5.0s)
  [SGD lr=0.1] ep   2/10 loss 1.8765 val_acc 0.3342 (5.0s)
  [SGD lr=0.1] ep   3/10 loss 1.7233 val_acc 0.3806 (5.0s)
  [SGD lr=0.1] ep   4/10 loss 1.6233 val_acc 0.3496 (5.0s)
  [SGD lr=0.1] ep   5/10 loss 1.5137 val_acc 0.4634 (5.0s)
  [SGD lr=0.1] ep   6/10 loss 1.4164 val_acc 0.5302 (5.0s)
  [SGD lr=0.1] ep   7/10 loss 1.3140 val_acc 0.4942 (5.0s)
  [SGD lr=0.1] ep   8/10 loss 1.2336 val_acc 0.5374 (5.0s)
  [SGD lr=0.1] ep   9/10 loss 1.1742 val_acc 0.5412 (5.0s)
  [SGD lr=0.1] ep  10/10 loss 1.1000 val_acc 0.5822 (5.0s)
[task2c_SGD+momentum_lr0.001: loaded saved run; its training log was:]
  [SGD+momentum lr=0.001] ep   1/10 loss 2.3029 val_acc 0.1054 (5.1s)
  [SGD+momentum lr=0.001] ep   2/10 loss 2.3010 val_acc 0.1482 (5.1s)
  [SGD+momentum lr=0.001] ep   3/10 loss 2.2969 val_acc 0.1320 (5.1s)
  [SGD+momentum lr=0.001] ep   4/10 loss 2.2787 val_acc 0.1178 (5.1s)
  [SGD+momentum lr=0.001] ep   5/10 loss 2.2247 val_acc 0.1664 (5.1s)
  [SGD+momentum lr=0.001] ep   6/10 loss 2.1438 val_acc 0.2280 (5.1s)
  [SGD+momentum lr=0.001] ep   7/10 loss 2.0366 val_acc 0.2692 (5.1s)
  [SGD+momentum lr=0.001] ep   8/10 loss 1.9301 val_acc 0.3152 (5.1s)
  [SGD+momentum lr=0.001] ep   9/10 loss 1.8329 val_acc 0.3378 (5.1s)
  [SGD+momentum lr=0.001] ep  10/10 loss 1.7640 val_acc 0.3508 (5.1s)
[task2c_SGD+momentum_lr0.01: loaded saved run; its training log was:]
  [SGD+momentum lr=0.01] ep   1/10 loss 2.1929 val_acc 0.3038 (5.1s)
  [SGD+momentum lr=0.01] ep   2/10 loss 1.7690 val_acc 0.3712 (5.1s)
  [SGD+momentum lr=0.01] ep   3/10 loss 1.5860 val_acc 0.3970 (5.1s)
  [SGD+momentum lr=0.01] ep   4/10 loss 1.4391 val_acc 0.4782 (5.1s)
  [SGD+momentum lr=0.01] ep   5/10 loss 1.2924 val_acc 0.5634 (5.1s)
  [SGD+momentum lr=0.01] ep   6/10 loss 1.1789 val_acc 0.6126 (5.1s)
  [SGD+momentum lr=0.01] ep   7/10 loss 1.0787 val_acc 0.6110 (5.1s)
  [SGD+momentum lr=0.01] ep   8/10 loss 1.0082 val_acc 0.6214 (5.1s)
  [SGD+momentum lr=0.01] ep   9/10 loss 0.9316 val_acc 0.6708 (5.1s)
  [SGD+momentum lr=0.01] ep  10/10 loss 0.8845 val_acc 0.6850 (5.1s)
```

```
[task2c_SGD+momentum_lr0.1: loaded saved run; its training log was:]
  [SGD+momentum lr=0.1] ep   1/10 loss 1.9672 val_acc 0.3152 (5.1s)
  [SGD+momentum lr=0.1] ep   2/10 loss 1.5052 val_acc 0.4562 (5.1s)
  [SGD+momentum lr=0.1] ep   3/10 loss 1.2371 val_acc 0.5884 (5.1s)
  [SGD+momentum lr=0.1] ep   4/10 loss 1.0571 val_acc 0.6624 (5.3s)
  [SGD+momentum lr=0.1] ep   5/10 loss 0.9384 val_acc 0.6436 (5.1s)
  [SGD+momentum lr=0.1] ep   6/10 loss 0.8546 val_acc 0.6644 (5.1s)
  [SGD+momentum lr=0.1] ep   7/10 loss 0.7947 val_acc 0.7000 (5.1s)
  [SGD+momentum lr=0.1] ep   8/10 loss 0.7425 val_acc 0.7244 (5.1s)
  [SGD+momentum lr=0.1] ep   9/10 loss 0.6884 val_acc 0.7138 (5.1s)
  [SGD+momentum lr=0.1] ep  10/10 loss 0.6722 val_acc 0.7226 (5.1s)
[task2c_Adam_lr0.001: loaded saved run; its training log was:]
  [Adam lr=0.001] ep   1/10 loss 1.7235 val_acc 0.4638 (5.2s)
  [Adam lr=0.001] ep   2/10 loss 1.3982 val_acc 0.5354 (5.2s)
  [Adam lr=0.001] ep   3/10 loss 1.2200 val_acc 0.5616 (5.2s)
  [Adam lr=0.001] ep   4/10 loss 1.0920 val_acc 0.6346 (5.2s)
  [Adam lr=0.001] ep   5/10 loss 0.9942 val_acc 0.6526 (5.2s)
  [Adam lr=0.001] ep   6/10 loss 0.9276 val_acc 0.6778 (5.2s)
  [Adam lr=0.001] ep   7/10 loss 0.8704 val_acc 0.6950 (5.2s)
  [Adam lr=0.001] ep   8/10 loss 0.8282 val_acc 0.6782 (5.2s)
  [Adam lr=0.001] ep   9/10 loss 0.7875 val_acc 0.7012 (5.2s)
  [Adam lr=0.001] ep  10/10 loss 0.7550 val_acc 0.7188 (5.2s)
[task2c_Adam_lr0.01: loaded saved run; its training log was:]
  [Adam lr=0.01] ep   1/10 loss 2.3125 val_acc 0.0934 (5.2s)
  [Adam lr=0.01] ep   2/10 loss 2.3032 val_acc 0.0934 (5.2s)
  [Adam lr=0.01] ep   3/10 loss 2.3034 val_acc 0.0996 (5.2s)
  [Adam lr=0.01] ep   4/10 loss 2.3033 val_acc 0.1008 (5.2s)
  [Adam lr=0.01] ep   5/10 loss 2.3032 val_acc 0.1008 (5.2s)
  [Adam lr=0.01] ep   6/10 loss 2.3032 val_acc 0.0996 (5.2s)
  [Adam lr=0.01] ep   7/10 loss 2.3033 val_acc 0.1008 (5.2s)
  [Adam lr=0.01] ep   8/10 loss 2.3033 val_acc 0.1016 (5.2s)
  [Adam lr=0.01] ep   9/10 loss 2.3034 val_acc 0.1012 (5.2s)
  [Adam lr=0.01] ep  10/10 loss 2.3033 val_acc 0.1012 (5.2s)
[task2c_Adam_lr0.1: loaded saved run; its training log was:]
  [Adam lr=0.1] ep   1/10 loss 567.2357 val_acc 0.1008 (5.2s)
  [Adam lr=0.1] ep   2/10 loss 2.3083 val_acc 0.0934 (5.2s)
  [Adam lr=0.1] ep   3/10 loss 2.3093 val_acc 0.1014 (5.2s)
  [Adam lr=0.1] ep   4/10 loss 2.3088 val_acc 0.0996 (5.2s)
  [Adam lr=0.1] ep   5/10 loss 2.3084 val_acc 0.1008 (5.2s)
  [Adam lr=0.1] ep   6/10 loss 2.3091 val_acc 0.0988 (5.2s)
  [Adam lr=0.1] ep   7/10 loss 2.3089 val_acc 0.0964 (5.2s)
  [Adam lr=0.1] ep   8/10 loss 2.3091 val_acc 0.1016 (5.2s)
  [Adam lr=0.1] ep   9/10 loss 2.3086 val_acc 0.1016 (5.2s)
  [Adam lr=0.1] ep  10/10 loss 2.3082 val_acc 0.1012 (5.2s)
```

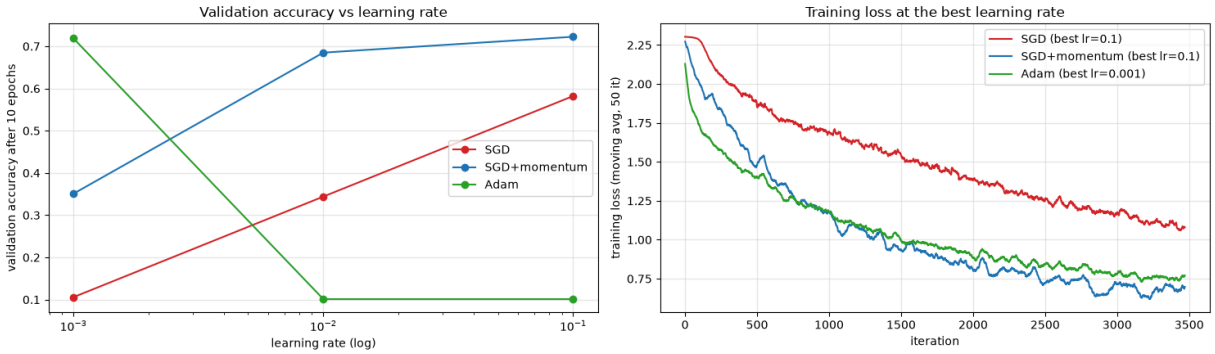
In [12]:

```python
def final_acc(H):
    return H["val_acc"][-1] if math.isfinite(H["train_loss"][-1]) else 0.1


fig, ax = plt.subplots(1, 2, figsize=(15, 4.5))
colors = dict(zip(OPTS_2C, ["tab:red", "tab:blue", "tab:green"]))
for o in OPTS_2C:
    accs = [final_acc(R2c[(o, lr)]) for lr in LRS_2C]
    ax[0].semilogx(LRS_2C, accs, "o-", color=colors[o], label=o)
    best = LRS_2C[int(np.argmax(accs))]
    ax[1].plot(smooth(R2c[(o, best)]["iter_loss"], 50), color=colors[o], label=f"
```

```
{o} (best lr={best:g})")
ax[0].set(xlabel="learning rate (log)", ylabel="validation accuracy after 10
epochs",
          title="Validation accuracy vs learning rate"); ax[0].grid(alpha=0.3);
ax[0].legend()
ax[1].set(xlabel="iteration", ylabel="training loss (moving avg, 50 it)",
          title="Training loss at the best learning rate")
ax[1].grid(alpha=0.3); ax[1].legend()
plt.tight_layout(); savefig("task2c_lr_sensitivity"); plt.show()

t2c = pd.DataFrame({f"lr = {lr:g}": [final_acc(R2c[(o, lr)]) for o in OPTS_2C] for
lr in LRS_2C}, index=list(OPTS_2C))
t2c["best lr"] = [f"{LRS_2C[int(np.argmax(row))]:g}" for row in t2c.values]
t2c["final train loss at best lr"] = [R2c[(o, float(b))]["train_loss"][-1] for o,
b in zip(OPTS_2C, t2c["best lr"])]
print("Final validation accuracy after 10 epochs:")
display(t2c.style.format({c: "{:.2%}" for c in t2c.columns if c.startswith("lr")}
|
                         {"final train loss at best lr": "{:.3f}"}).highlight_max(
    axis=None, subset=[c for c in t2c.columns if c.startswith("lr")],
color="#c6efce"))
```



Final validation accuracy after 10 epochs:

	lr = 0.001	lr = 0.01	lr = 0.1	best lr	final train loss at best lr
SGD	10.58%	34.40%	58.22%	0.1	1.100
SGD+momentum	35.08%	68.50%	72.26%	0.1	0.672
Adam	71.88%	10.12%	10.12%	0.001	0.755

Discussion of sensitivity. Final validation accuracy after 10 epochs:

optimizer	lr = 0.001	lr = 0.01	lr = 0.1
SGD	10.6%	34.4%	**58.2%**
SGD + momentum 0.9	35.1%	68.5%	**72.3%**
Adam	**71.9%**	10.1% (stuck)	10.1% (diverged, loss up to 2×10^5)

- **Plain SGD** needs a large learning rate: at 10^{-3} it does not leave the initial plateau (loss stays at 2.30) within 10 epochs, and accuracy increases monotonically with the learning rate over the grid, i.e. the best value may even lie above 0.1. It is the slowest optimizer (the training loss at its best rate is still 1.10).

- **Momentum** behaves like SGD with a 10× larger effective learning rate $\eta/(1-\mu)$: momentum at η matches plain SGD at 10η (35.1% vs 34.4%; 68.5% vs 58.2%). It is good over a wider range (68–72% for $\eta \in [0.01, 0.1]$) and gives the lowest training loss and best accuracy at $\eta = 0.1$.
- **Adam** is the fastest at the start (the steepest loss decrease in the right plot, and the best first epoch), but it is the *most sensitive* optimizer: it only works at 10^{-3}. Because Adam's step size is about η per parameter regardless of the gradient scale, $\eta = 10^{-2}$ already moves every weight by 0.01 per step, which destroys the initial features of this network without batch norm (the ReLUs die and the loss sticks at $\ln 10 = 2.30$), and $\eta = 0.1$ makes the loss explode.

So the "best" learning rate differs by two orders of magnitude between optimizers (0.1 for SGD/momentum, 0.001 for Adam) and must be tuned per optimizer; comparing optimizers at a single shared learning rate would be misleading. With its best rate, SGD with momentum slightly outperforms Adam in final accuracy here, while Adam is the easiest to get working quickly with its default 10^{-3}.

Task 3: Regularization and residual connections

(a) Regularization on a 5k-image training subset

Model: the same small CNN as in 2(c) with batch norm after every convolution (dropout layers are present but only active when $p > 0$). Training data: the first 5,000 images of the 45k training split (classes roughly balanced); validation: the usual 5k validation split. All runs use the same seed, SGD with momentum 0.9, peak learning rate 0.05 with cosine decay, batch size 128 and 100 epochs. Train accuracy is measured on the *clean* (non-augmented) 5k training images in eval mode after every epoch, so it is comparable across methods.

variant	setting
baseline	no regularization
weight decay	L2 penalty 5×10^{-4} (coupled, via the optimizer)
dropout	$p = 0.3$ after each pooling stage and before the classifier
flip + crop	random horizontal flip + random 32×32 crop of the 4-pixel padded image
cutout	one random 16×16 square zeroed per image (own implementation, `cutout` above)
mixup	$\tilde{x} = \lambda x_i + (1-\lambda) x_j$, loss $\lambda\,\mathrm{CE}(y_i) + (1-\lambda)\,\mathrm{CE}(y_j)$, $\lambda \sim \mathrm{Beta}(0.4, 0.4)$ (own implementation, `mixup` above)
all combined	all of the above together

In [13]:

```
X_small, y_small = X_tr[:5000], y_tr[:5000]
print("5k subset class counts:", torch.bincount(y_small.cpu()).tolist())

VARIANTS_3A = {
    "baseline": dict(),
    "weight decay": dict(wd=5e-4),
    "dropout": dict(p_drop=0.3),
    "flip+crop": dict(aug=flip_crop),
    "cutout": dict(aug=cutout),
    "mixup": dict(mixup_alpha=0.4),
    "all combined": dict(wd=5e-4, p_drop=0.3, aug=compose(flip_crop, cutout),
mixup_alpha=0.4),
}


def run_3a(name, n_epochs=100):
    cfg = VARIANTS_3A[name]
    set_seed()
    model = SmallCNN(bn=True, p_drop=cfg.get("p_drop", 0.0)).to(device)
    opt = MySGD(model.parameters(), lr=0.05, momentum=0.9,
weight_decay=cfg.get("wd", 0.0))
    steps = n_epochs * math.ceil(len(y_small) / 128)
    return fit(model, opt, n_epochs, X_small, y_small, bs=128, aug=cfg.get("aug"),
               mixup_alpha=cfg.get("mixup_alpha", 0.0), sched=cosine_warmup(opt,
steps), eval_train=True,
               tag=name)


R3a = {k: cached(f"task3a_{k.replace(' ', '_').replace('+', '_')}", lambda k=k:
run_3a(k, epochs(100)))
       for k in VARIANTS_3A}
```

```
5k subset class counts: [511, 510, 506, 509, 502, 472, 500, 470, 500, 520]
[task3a_baseline: loaded saved run; its training log was:]
  [baseline] ep   1/100 loss 1.8772 val_acc 0.2764 train_acc 0.2906 (15.6s)
  [baseline] ep  10/100 loss 0.9129 val_acc 0.4872 train_acc 0.5562 (3.1s)
  [baseline] ep  20/100 loss 0.3398 val_acc 0.6164 train_acc 0.8358 (3.1s)
  [baseline] ep  30/100 loss 0.1767 val_acc 0.6224 train_acc 0.9300 (3.1s)
  [baseline] ep  40/100 loss 0.1188 val_acc 0.6326 train_acc 0.9600 (3.1s)
  [baseline] ep  50/100 loss 0.0296 val_acc 0.6306 train_acc 0.9606 (3.1s)
  [baseline] ep  60/100 loss 0.0096 val_acc 0.6826 train_acc 1.0000 (3.1s)
  [baseline] ep  70/100 loss 0.0084 val_acc 0.6806 train_acc 1.0000 (3.1s)
  [baseline] ep  80/100 loss 0.0098 val_acc 0.6770 train_acc 1.0000 (3.1s)
  [baseline] ep  90/100 loss 0.0089 val_acc 0.6832 train_acc 1.0000 (3.1s)
  [baseline] ep 100/100 loss 0.0088 val_acc 0.6818 train_acc 1.0000 (3.1s)
[task3a_weight_decay: loaded saved run; its training log was:]
  [weight decay] ep   1/100 loss 1.8744 val_acc 0.1948 train_acc 0.2076 (3.1s)
  [weight decay] ep  10/100 loss 0.9319 val_acc 0.4756 train_acc 0.5416 (3.1s)
  [weight decay] ep  20/100 loss 0.3781 val_acc 0.5636 train_acc 0.7862 (3.1s)
  [weight decay] ep  30/100 loss 0.1919 val_acc 0.6470 train_acc 0.9520 (3.1s)
  [weight decay] ep  40/100 loss 0.1171 val_acc 0.6270 train_acc 0.9568 (3.1s)
  [weight decay] ep  50/100 loss 0.0207 val_acc 0.6822 train_acc 1.0000 (3.1s)
  [weight decay] ep  60/100 loss 0.0100 val_acc 0.6778 train_acc 1.0000 (3.1s)
  [weight decay] ep  70/100 loss 0.0168 val_acc 0.6744 train_acc 1.0000 (3.1s)
  [weight decay] ep  80/100 loss 0.0066 val_acc 0.6792 train_acc 1.0000 (3.1s)
  [weight decay] ep  90/100 loss 0.0113 val_acc 0.6802 train_acc 1.0000 (3.1s)
  [weight decay] ep 100/100 loss 0.0129 val_acc 0.6790 train_acc 1.0000 (3.1s)
[task3a_dropout: loaded saved run; its training log was:]
  [dropout] ep   1/100 loss 2.0229 val_acc 0.2190 train_acc 0.2238 (3.1s)
  [dropout] ep  10/100 loss 1.3257 val_acc 0.5016 train_acc 0.5252 (3.1s)
  [dropout] ep  20/100 loss 1.0154 val_acc 0.5788 train_acc 0.6476 (3.1s)
  [dropout] ep  30/100 loss 0.8561 val_acc 0.6342 train_acc 0.7668 (3.1s)
  [dropout] ep  40/100 loss 0.7056 val_acc 0.6480 train_acc 0.8058 (3.1s)
  [dropout] ep  50/100 loss 0.5596 val_acc 0.6580 train_acc 0.8500 (3.1s)
  [dropout] ep  60/100 loss 0.4986 val_acc 0.6790 train_acc 0.9154 (3.1s)
  [dropout] ep  70/100 loss 0.3921 val_acc 0.6812 train_acc 0.9454 (3.1s)
  [dropout] ep  80/100 loss 0.3359 val_acc 0.6958 train_acc 0.9700 (3.1s)
  [dropout] ep  90/100 loss 0.2946 val_acc 0.6996 train_acc 0.9810 (3.1s)
  [dropout] ep 100/100 loss 0.2738 val_acc 0.7010 train_acc 0.9820 (3.1s)
[task3a_flip_crop: loaded saved run; its training log was:]
  [flip+crop] ep   1/100 loss 1.9330 val_acc 0.2236 train_acc 0.2352 (3.1s)
  [flip+crop] ep  10/100 loss 1.1583 val_acc 0.4868 train_acc 0.5142 (3.1s)
  [flip+crop] ep  20/100 loss 0.8471 val_acc 0.6330 train_acc 0.7096 (3.1s)
  [flip+crop] ep  30/100 loss 0.7165 val_acc 0.6234 train_acc 0.7068 (3.1s)
  [flip+crop] ep  40/100 loss 0.5196 val_acc 0.6914 train_acc 0.8206 (3.1s)
  [flip+crop] ep  50/100 loss 0.4720 val_acc 0.7058 train_acc 0.8614 (3.1s)
  [flip+crop] ep  60/100 loss 0.3209 val_acc 0.7002 train_acc 0.8782 (3.1s)
  [flip+crop] ep  70/100 loss 0.2544 val_acc 0.7282 train_acc 0.9254 (3.1s)
  [flip+crop] ep  80/100 loss 0.1868 val_acc 0.7378 train_acc 0.9622 (3.1s)
  [flip+crop] ep  90/100 loss 0.1660 val_acc 0.7422 train_acc 0.9678 (3.1s)
  [flip+crop] ep 100/100 loss 0.1664 val_acc 0.7394 train_acc 0.9680 (3.1s)
[task3a_cutout: loaded saved run; its training log was:]
  [cutout] ep   1/100 loss 1.9516 val_acc 0.2094 train_acc 0.2240 (3.1s)
  [cutout] ep  10/100 loss 1.1588 val_acc 0.5576 train_acc 0.6256 (3.1s)
  [cutout] ep  20/100 loss 0.9291 val_acc 0.5586 train_acc 0.6602 (3.1s)
  [cutout] ep  30/100 loss 0.6297 val_acc 0.5938 train_acc 0.7548 (3.1s)
  [cutout] ep  40/100 loss 0.4569 val_acc 0.5670 train_acc 0.7800 (3.1s)
  [cutout] ep  50/100 loss 0.3436 val_acc 0.6758 train_acc 0.9814 (3.1s)
```

```
  [cutout] ep  60/100 loss 0.2255 val_acc 0.6898 train_acc 0.9946 (3.1s)
  [cutout] ep  70/100 loss 0.1212 val_acc 0.6932 train_acc 0.9984 (3.1s)
  [cutout] ep  80/100 loss 0.0909 val_acc 0.7044 train_acc 0.9998 (3.1s)
  [cutout] ep  90/100 loss 0.0732 val_acc 0.7068 train_acc 1.0000 (3.1s)
  [cutout] ep 100/100 loss 0.0679 val_acc 0.7108 train_acc 1.0000 (3.1s)
[task3a_mixup: loaded saved run; its training log was:]
  [mixup] ep   1/100 loss 2.0170 val_acc 0.2322 train_acc 0.2416 (3.1s)
  [mixup] ep  10/100 loss 1.5483 val_acc 0.5348 train_acc 0.5964 (3.1s)
  [mixup] ep  20/100 loss 1.1864 val_acc 0.5788 train_acc 0.7316 (3.1s)
  [mixup] ep  30/100 loss 0.8939 val_acc 0.5618 train_acc 0.7788 (3.1s)
  [mixup] ep  40/100 loss 0.8063 val_acc 0.6450 train_acc 0.9682 (3.1s)
  [mixup] ep  50/100 loss 0.7492 val_acc 0.6114 train_acc 0.9738 (3.1s)
  [mixup] ep  60/100 loss 0.7123 val_acc 0.6690 train_acc 0.9996 (3.1s)
  [mixup] ep  70/100 loss 0.7283 val_acc 0.6740 train_acc 1.0000 (3.1s)
  [mixup] ep  80/100 loss 0.4890 val_acc 0.6896 train_acc 1.0000 (3.1s)
  [mixup] ep  90/100 loss 0.6846 val_acc 0.6816 train_acc 1.0000 (3.1s)
  [mixup] ep 100/100 loss 0.6056 val_acc 0.6860 train_acc 1.0000 (3.1s)
[task3a_all_combined: loaded saved run; its training log was:]
  [all combined] ep   1/100 loss 2.1918 val_acc 0.1856 train_acc 0.1888 (3.1s)
  [all combined] ep  10/100 loss 1.8722 val_acc 0.4184 train_acc 0.4202 (3.1s)
  [all combined] ep  20/100 loss 1.7135 val_acc 0.4970 train_acc 0.5172 (3.1s)
  [all combined] ep  30/100 loss 1.5614 val_acc 0.5512 train_acc 0.5838 (3.1s)
  [all combined] ep  40/100 loss 1.5189 val_acc 0.5982 train_acc 0.6580 (3.2s)
  [all combined] ep  50/100 loss 1.4852 val_acc 0.6178 train_acc 0.6722 (3.2s)
  [all combined] ep  60/100 loss 1.4528 val_acc 0.6418 train_acc 0.7180 (3.2s)
  [all combined] ep  70/100 loss 1.4433 val_acc 0.6398 train_acc 0.7112 (3.2s)
  [all combined] ep  80/100 loss 1.2910 val_acc 0.6746 train_acc 0.7578 (3.2s)
  [all combined] ep  90/100 loss 1.4022 val_acc 0.6672 train_acc 0.7590 (3.2s)
  [all combined] ep 100/100 loss 1.3265 val_acc 0.6700 train_acc 0.7618 (3.2s)
```

In [14]:

```
t3a = pd.DataFrame({k: {"train acc": H["train_acc"][-1], "val acc": H["val_acc"]
[-1],
                        "gap (train - val)": H["train_acc"][-1] - H["val_acc"]
[-1],
                        "best val acc": max(H["val_acc"]), "final val loss":
H["val_loss"][-1]}
                    for k, H in R3a.items()}).T
print("Final epoch (100) on the 5k-image training subset:")
display(t3a.style.format({c: "{:.2%}" for c in t3a.columns if c != "final val
loss"} | {"final val loss": "{:.3f}"})
        .highlight_max(subset=["val acc"], color="#c6efce").highlight_min(subset=
["gap (train - val)"], color="#c6efce"))

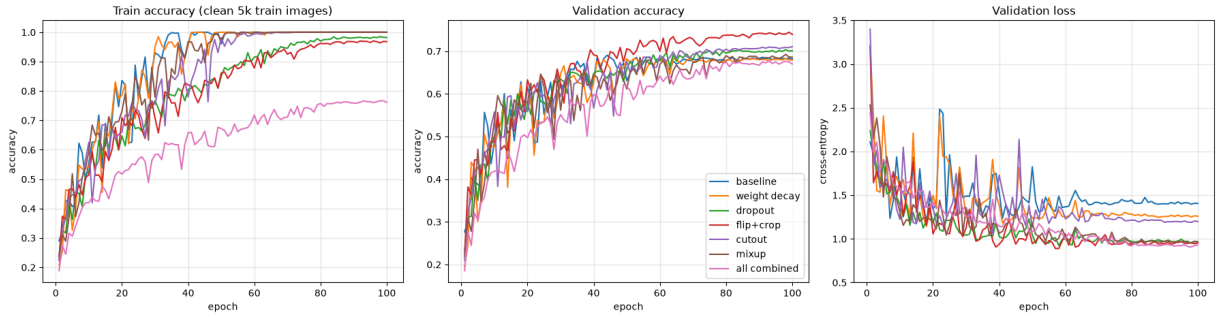
fig, axes = plt.subplots(1, 3, figsize=(18, 4.8))
cmap = plt.get_cmap("tab10")
for i, (k, H) in enumerate(R3a.items()):
    ep = np.arange(1, len(H["val_acc"]) + 1)
    axes[0].plot(ep, H["train_acc"], color=cmap(i), label=k)
    axes[1].plot(ep, H["val_acc"], color=cmap(i), label=k)
    axes[2].plot(ep, H["val_loss"], color=cmap(i), label=k)
axes[0].set(xlabel="epoch", ylabel="accuracy", title="Train accuracy (clean 5k
train images)")
axes[1].set(xlabel="epoch", ylabel="accuracy", title="Validation accuracy")
axes[2].set(xlabel="epoch", ylabel="cross-entropy", title="Validation loss", ylim=
(0.5, 3.5))
for a in axes:
```

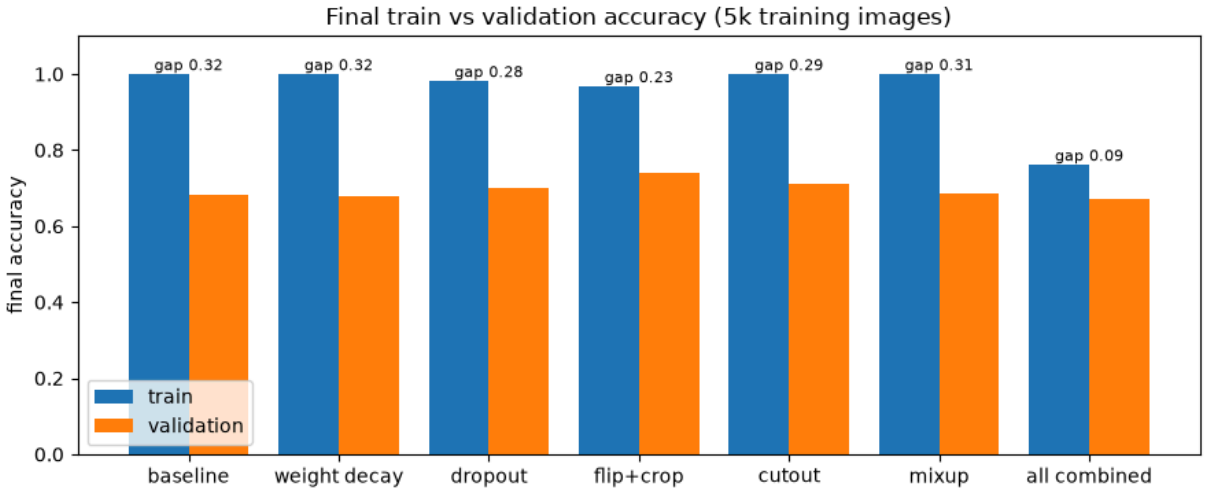
```
    a.grid(alpha=0.3)
axes[1].legend(loc="lower right")
plt.tight_layout(); savefig("task3a_regularization"); plt.show()

fig, ax = plt.subplots(figsize=(9, 3.8))
names = list(R3a)
tr = [R3a[k]["train_acc"][-1] for k in names]; va = [R3a[k]["val_acc"][-1] for k
in names]
xpos = np.arange(len(names))
ax.bar(xpos - 0.2, tr, 0.4, label="train"); ax.bar(xpos + 0.2, va, 0.4,
label="validation")
for x_, t_, v_ in zip(xpos, tr, va):
    ax.text(x_, max(t_, v_) + 0.01, f"gap {t_ - v_:.2f}", ha="center", fontsize=8)
ax.set_xticks(xpos); ax.set_xticklabels(names); ax.set_ylim(0, 1.1);
ax.set_ylabel("final accuracy")
ax.set_title("Final train vs validation accuracy (5k training images)");
ax.legend(loc="lower left")
plt.tight_layout(); savefig("task3a_gap"); plt.show()
```

Final epoch (100) on the 5k-image training subset:

	train acc	val acc	gap (train - val)	best val acc	final val loss
baseline	100.00%	68.18%	31.82%	69.04%	1.406
weight decay	100.00%	67.90%	32.10%	68.44%	1.259
dropout	98.20%	70.10%	28.10%	70.36%	0.967
flip+crop	96.80%	73.94%	22.86%	74.44%	0.953
cutout	100.00%	71.08%	28.92%	71.08%	1.199
mixup	100.00%	68.60%	31.40%	69.26%	0.965
all combined	76.18%	67.00%	9.18%	67.70%	0.929





Interpretation. Results after 100 epochs on 5k images (train accuracy on clean training images):

variant	train acc	val acc	gap	final val loss
baseline	100.0%	68.2%	31.8	1.41
weight decay	100.0%	67.9%	32.1	1.26
dropout	98.2%	70.1%	28.1	0.97
flip + crop	96.8%	**73.9%**	22.9	0.95
cutout	100.0%	71.1%	28.9	1.20
mixup	100.0%	68.6%	31.4	0.97
all combined	76.2%	67.0%	**9.2**	**0.93**

- With only 5k images the **baseline** memorises the training set within ~35 epochs (100% train accuracy) and stalls at 68% validation accuracy: a 32-point generalisation gap. Its validation loss is also the highest and very noisy, because the network becomes over-confident.
- **Flip + crop** is the most effective single regulariser (+5.8 points, smallest gap among single methods): it encodes real invariances of natural images (left–right symmetry, small translations), effectively multiplying the size of the training set. **Cutout** (+2.9) forces the network to use several parts of the object instead of a single discriminative patch, but on its own the network still reaches 100% on clean training images. **Dropout** (+1.9) reduces co-adaptation of features and is the only single method besides flip+crop that keeps train accuracy below 100%.
- **Weight decay** barely changes accuracy here. With batch norm after every conv the network output is invariant to the scale of the conv weights, so L2 mainly increases the effective learning rate rather than restricting capacity; it does lower the validation loss (1.41 → 1.26). **Mixup** gives a similar accuracy to the baseline, but the validation loss drops from 1.41 to 0.97: training on interpolated images with soft targets makes the predictions much better calibrated (less over-confident), even though the arg-max decisions change little.

- **All combined** has by far the smallest gap (9 points) and the lowest validation loss, but it **underfits**: the train accuracy only reaches 76% and validation accuracy (67%) is lower than with flip+crop alone. Stacking five regularisers on a 0.14M-parameter network trained for only 4k iterations removes too much effective capacity; the curves are still rising at epoch 100, so a bigger model or a longer schedule would be needed to benefit from it.

In short, regularisation trades training fit for generalisation; the best choice depends on the capacity/data/compute balance, and data augmentation that reflects true invariances (flip + crop) is the most valuable when data is scarce.

(b) Plain vs residual CIFAR networks of depth $6n + 2$ (8, 20, 56)

Architecture of He et al. (2016) for CIFAR: a 3×3 conv with 16 channels, then three stages of n basic blocks (two 3×3 conv–BN layers each) with 16/32/64 channels at resolutions 32/16/8 (the first conv of stages 2 and 3 has stride 2), global average pooling and a 10-way FC layer; $6n + 2$ weighted layers in total. Kaiming-normal initialization (fan-out, ReLU) for all convolutions, BN with $\gamma = 1, \beta = 0$.

The **only** difference between the two families is the shortcut: residual blocks compute $\mathrm{ReLU}(F(x) + s(x))$ and plain blocks $\mathrm{ReLU}(F(x))$. We use the parameter-free option-A shortcut (identity; when the shape changes, stride-2 subsampling and zero-padding of the extra channels), so plain and residual networks of the same depth have *exactly* the same parameters.

Training (identical for all 6 networks): SGD, momentum 0.9, weight decay 10^{-4}, batch 128, peak learning rate 0.1 with 1 epoch of linear warm-up and cosine decay, flip + crop augmentation, 20 epochs on the 45k split (fp32 on the GTX 1650; fp16 mixed precision is used automatically on tensor-core GPUs such as the T4).

In [15]:
```
class ShortcutA(nn.Module):
    """Identity shortcut; on a shape change, subsample spatially and zero-pad the
new channels."""
    def __init__(self, c_in, c_out, stride):
        super().__init__()
        self.stride, self.pad = stride, c_out - c_in

    def forward(self, x):
        if self.stride > 1:
            x = x[:, :, ::self.stride, ::self.stride]
        if self.pad > 0:
            x = F.pad(x, (0, 0, 0, 0, self.pad // 2, self.pad - self.pad // 2))
        return x


class ShortcutB(nn.Sequential):
    """Projection shortcut (1x1 conv + BN) on a shape change, identity
```

```
otherwise."""
    def __init__(self, c_in, c_out, stride):
        if stride == 1 and c_in == c_out:
            super().__init__()
        else:
            super().__init__(nn.Conv2d(c_in, c_out, 1, stride, bias=False),
nn.BatchNorm2d(c_out))


class BasicBlock(nn.Module):
    def __init__(self, c_in, c_out, stride, residual=True, shortcut="A"):
        super().__init__()
        self.conv1 = nn.Conv2d(c_in, c_out, 3, stride, 1, bias=False)
        self.bn1 = nn.BatchNorm2d(c_out)
        self.conv2 = nn.Conv2d(c_out, c_out, 3, 1, 1, bias=False)
        self.bn2 = nn.BatchNorm2d(c_out)
        self.residual = residual
        if residual:
            self.shortcut = (ShortcutA if shortcut == "A" else ShortcutB)(c_in,
c_out, stride)

    def forward(self, x):
        out = F.relu(self.bn1(self.conv1(x)))
        out = self.bn2(self.conv2(out))
        if self.residual:
            out = out + self.shortcut(x)
        return F.relu(out)


class CifarNet(nn.Module):
    """CIFAR ResNet / plain net: stem + 3 stages + GAP + FC. blocks=(n, n, n)
gives depth 6n+2."""
    def __init__(self, blocks=(3, 3, 3), widths=(16, 32, 64), residual=True,
shortcut="A", zero_init_last_bn=False):
        super().__init__()
        self.stem = nn.Sequential(nn.Conv2d(3, widths[0], 3, 1, 1, bias=False),
nn.BatchNorm2d(widths[0]),
                                  nn.ReLU(inplace=True))
        layers, c_in = [], widths[0]
        for s, (n, w) in enumerate(zip(blocks, widths)):
            for b in range(n):
                layers.append(BasicBlock(c_in, w, 2 if (s > 0 and b == 0) else 1,
residual, shortcut))
                c_in = w
        self.stages = nn.Sequential(*layers)
        self.fc = nn.Linear(c_in, 10)
        for m in self.modules():
            if isinstance(m, nn.Conv2d):
                nn.init.kaiming_normal_(m.weight, mode="fan_out",
nonlinearity="relu")
            elif isinstance(m, nn.BatchNorm2d):
                nn.init.ones_(m.weight); nn.init.zeros_(m.bias)
            elif isinstance(m, nn.Linear):
                nn.init.kaiming_normal_(m.weight); nn.init.zeros_(m.bias)
        if zero_init_last_bn and residual:
            for m in self.modules():
```

```
                if isinstance(m, BasicBlock):
                    nn.init.zeros_(m.bn2.weight)

    def forward(self, x):
        x = self.stages(self.stem(x))
        return self.fc(torch.flatten(F.adaptive_avg_pool2d(x, 1), 1))


def depth_net(depth, residual):
    n = (depth - 2) // 6
    return CifarNet(blocks=(n, n, n), residual=residual)


for d in (8, 20, 56):
    print(f"depth {d:2d}: params plain {n_params(depth_net(d, False)):,}  residual
{n_params(depth_net(d, True)):,}")
```

```
depth  8: params plain 75,290  residual 75,290
depth 20: params plain 269,722  residual 269,722
depth 56: params plain 853,018  residual 853,018
```

In [16]:
```
def run_3b(depth, residual, n_epochs=20):
    set_seed()
    model = depth_net(depth, residual).to(device)
    opt = MySGD(model.parameters(), lr=0.1, momentum=0.9, weight_decay=1e-4)
    spe = math.ceil(len(y_tr) / 128)
    return fit(model, opt, n_epochs, X_tr, y_tr, bs=128, aug=flip_crop,
               sched=cosine_warmup(opt, n_epochs * spe, warmup_steps=spe),
               tag=f"{'res' if residual else 'plain'}-{depth}")


R3b = {(d, r): cached(f"task3b_{'res' if r else 'plain'}{d}", lambda d=d, r=r:
run_3b(d, r, epochs(20)))
       for d in (8, 20, 56) for r in (False, True)}
```

```
[plain-8] ep   1/20 loss 1.7175 val_acc 0.3700 (6.6s)
[plain-8] ep   2/20 loss 1.2060 val_acc 0.4112 (4.9s)
[plain-8] ep   4/20 loss 0.9305 val_acc 0.6416 (4.9s)
[plain-8] ep   6/20 loss 0.7973 val_acc 0.7008 (4.9s)
[plain-8] ep   8/20 loss 0.6970 val_acc 0.7340 (5.0s)
[plain-8] ep  10/20 loss 0.6266 val_acc 0.7678 (4.9s)
[plain-8] ep  12/20 loss 0.5706 val_acc 0.7922 (4.9s)
[plain-8] ep  14/20 loss 0.5251 val_acc 0.8038 (4.9s)
[plain-8] ep  16/20 loss 0.4880 val_acc 0.8170 (4.9s)
[plain-8] ep  18/20 loss 0.4634 val_acc 0.8284 (4.9s)
[plain-8] ep  20/20 loss 0.4495 val_acc 0.8294 (4.9s)
[res-8] ep   1/20 loss 1.7412 val_acc 0.4592 (5.3s)
[res-8] ep   2/20 loss 1.2452 val_acc 0.5592 (5.3s)
[res-8] ep   4/20 loss 0.9396 val_acc 0.5686 (5.3s)
[res-8] ep   6/20 loss 0.8013 val_acc 0.6724 (5.3s)
[res-8] ep   8/20 loss 0.7190 val_acc 0.7274 (5.3s)
[res-8] ep  10/20 loss 0.6539 val_acc 0.7548 (5.2s)
[res-8] ep  12/20 loss 0.5968 val_acc 0.7842 (5.3s)
[res-8] ep  14/20 loss 0.5513 val_acc 0.7936 (5.2s)
[res-8] ep  16/20 loss 0.5133 val_acc 0.8220 (5.3s)
[res-8] ep  18/20 loss 0.4891 val_acc 0.8286 (5.3s)
[res-8] ep  20/20 loss 0.4767 val_acc 0.8304 (5.3s)
```

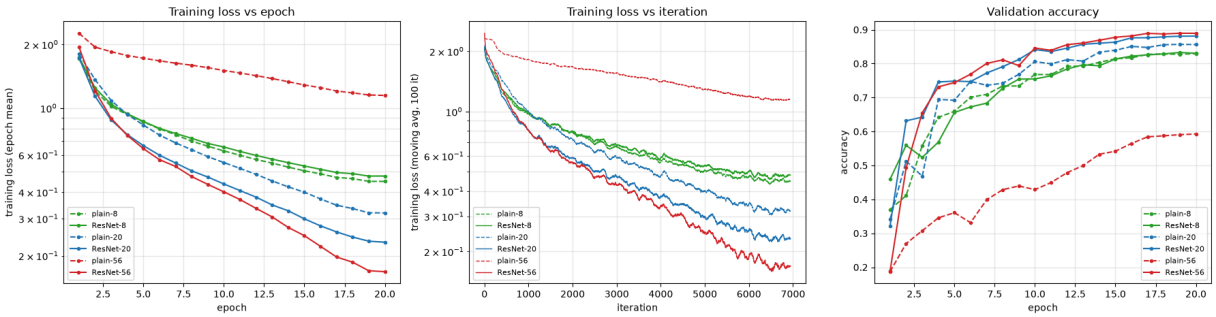
```
[plain-20] ep   1/20 loss 1.8075 val_acc 0.3412 (13.3s)
[plain-20] ep   2/20 loss 1.3610 val_acc 0.5128 (13.3s)
[plain-20] ep   4/20 loss 0.9322 val_acc 0.6944 (13.4s)
[plain-20] ep   6/20 loss 0.7447 val_acc 0.7476 (13.4s)
[plain-20] ep   8/20 loss 0.6346 val_acc 0.7422 (13.4s)
[plain-20] ep  10/20 loss 0.5520 val_acc 0.8060 (13.5s)
[plain-20] ep  12/20 loss 0.4856 val_acc 0.8116 (13.6s)
[plain-20] ep  14/20 loss 0.4236 val_acc 0.8338 (13.4s)
[plain-20] ep  16/20 loss 0.3715 val_acc 0.8506 (13.5s)
[plain-20] ep  18/20 loss 0.3345 val_acc 0.8556 (13.4s)
[plain-20] ep  20/20 loss 0.3187 val_acc 0.8560 (13.3s)
[res-20] ep   1/20 loss 1.7330 val_acc 0.3216 (14.1s)
[res-20] ep   2/20 loss 1.1359 val_acc 0.6314 (14.1s)
[res-20] ep   4/20 loss 0.7490 val_acc 0.7460 (14.1s)
[res-20] ep   6/20 loss 0.5984 val_acc 0.7470 (14.1s)
[res-20] ep   8/20 loss 0.5044 val_acc 0.7910 (14.1s)
[res-20] ep  10/20 loss 0.4373 val_acc 0.8412 (14.1s)
[res-20] ep  12/20 loss 0.3787 val_acc 0.8456 (14.1s)
[res-20] ep  14/20 loss 0.3265 val_acc 0.8600 (14.2s)
[res-20] ep  16/20 loss 0.2770 val_acc 0.8758 (14.1s)
[res-20] ep  18/20 loss 0.2451 val_acc 0.8788 (14.2s)
[res-20] ep  20/20 loss 0.2317 val_acc 0.8810 (14.1s)
[plain-56] ep   1/20 loss 2.2587 val_acc 0.1870 (39.1s)
[plain-56] ep   2/20 loss 1.9416 val_acc 0.2698 (39.1s)
[plain-56] ep   4/20 loss 1.7712 val_acc 0.3456 (39.1s)
[plain-56] ep   6/20 loss 1.6722 val_acc 0.3314 (39.1s)
[plain-56] ep   8/20 loss 1.5948 val_acc 0.4286 (39.4s)
[plain-56] ep  10/20 loss 1.5054 val_acc 0.4288 (39.4s)
[plain-56] ep  12/20 loss 1.4228 val_acc 0.4788 (39.3s)
[plain-56] ep  14/20 loss 1.3320 val_acc 0.5330 (39.3s)
[plain-56] ep  16/20 loss 1.2483 val_acc 0.5648 (39.5s)
[plain-56] ep  18/20 loss 1.1796 val_acc 0.5872 (39.3s)
[plain-56] ep  20/20 loss 1.1468 val_acc 0.5926 (39.2s)
[res-56] ep   1/20 loss 1.9436 val_acc 0.1884 (41.3s)
[res-56] ep   2/20 loss 1.2051 val_acc 0.4934 (41.0s)
[res-56] ep   4/20 loss 0.7420 val_acc 0.7316 (41.2s)
[res-56] ep   6/20 loss 0.5695 val_acc 0.7688 (41.1s)
[res-56] ep   8/20 loss 0.4738 val_acc 0.8110 (40.9s)
[res-56] ep  10/20 loss 0.4012 val_acc 0.8458 (41.0s)
[res-56] ep  12/20 loss 0.3351 val_acc 0.8558 (41.0s)
[res-56] ep  14/20 loss 0.2723 val_acc 0.8688 (41.0s)
[res-56] ep  16/20 loss 0.2216 val_acc 0.8816 (41.0s)
[res-56] ep  18/20 loss 0.1864 val_acc 0.8872 (41.0s)
[res-56] ep  20/20 loss 0.1682 val_acc 0.8892 (41.0s)
```

In [17]:

```
fig, ax = plt.subplots(1, 3, figsize=(18, 4.8))
dcol = {8: "tab:green", 20: "tab:blue", 56: "tab:red"}
for (d, r), H in R3b.items():
    ep = np.arange(1, len(H["train_loss"]) + 1)
    style = dict(color=dcol[d], ls="-" if r else "--", label=f"{'ResNet' if r else
'plain'}-{d}")
    ax[0].plot(ep, H["train_loss"], marker="o", ms=3, **style)
    ax[1].plot(smooth(H["iter_loss"], 100), lw=1, **style)
    ax[2].plot(ep, H["val_acc"], marker="o", ms=3, **style)
ax[0].set(xlabel="epoch", ylabel="training loss (epoch mean)", title="Training
loss vs epoch", yscale="log")
```

```
ax[1].set(xlabel="iteration", ylabel="training loss (moving avg, 100 it)",
title="Training loss vs iteration",
          yscale="log")
ax[2].set(xlabel="epoch", ylabel="accuracy", title="Validation accuracy")
for a in ax:
    a.grid(alpha=0.3, which="both"); a.legend(fontsize=8)
plt.tight_layout(); savefig("task3b_plain_vs_residual"); plt.show()

t3b = pd.DataFrame([{"depth": d, "parameters": n_params(depth_net(d, True)),
                     "plain: train loss": R3b[(d, False)]["train_loss"][-1],
                     "residual: train loss": R3b[(d, True)]["train_loss"][-1],
                     "plain: val acc": R3b[(d, False)]["val_acc"][-1],
                     "residual: val acc": R3b[(d, True)]["val_acc"][-1],
                     "plain: s/epoch": np.mean(R3b[(d, False)]["time"]),
                     "residual: s/epoch": np.mean(R3b[(d, True)]["time"])} for d
in (8, 20, 56)]).set_index("depth")
print("Final epoch (20):")
display(t3b.style.format({"parameters": "{:,}"} | {c: "{:.4f}" for c in
t3b.columns if "loss" in c}
                         | {c: "{:.2%}" for c in t3b.columns if "acc" in c}
                         | {c: "{:.1f}" for c in t3b.columns if "s/epoch" in c}))
```




```
Final epoch (20):
```

depth	parameters	plain: train loss	residual: train loss	plain: val acc	residual: val acc	plain: s/epoch	residual: s/epoch
8	75,290	0.4495	0.4767	82.94%	83.04%	5.0	5.3
20	269,722	0.3187	0.2317	85.60%	88.10%	13.4	14.1
56	853,018	1.1468	0.1682	59.26%	88.92%	39.3	41.1

Explanation. Final training loss / validation accuracy after 20 epochs:

depth	plain	residual
8	0.450 / 82.9%	0.477 / 83.0%
20	0.319 / 85.6%	0.232 / 88.1%
56	**1.147 / 59.3%**	**0.168 / 88.9%**

- **Plain networks degrade with depth.** Going from 8 to 20 layers still helps a little, but the 56-layer plain network is dramatically *worse*: its training loss after 20 epochs (1.15) is 2.5× that of the 8-layer plain network, and its loss curve decreases only slowly from the start. Because it is the **training** loss that is higher (and the validation accuracy is lower

by the same amount), this is not overfitting but an **optimization problem**: a 56-layer network can represent everything the 8-layer one can (extra layers could learn the identity), yet SGD fails to find such a solution. This is the *degradation problem* of He et al. Even with batch norm and Kaiming initialization (so the gradients do not simply vanish at initialization, cf. Task 1(d)), the composition of 54 non-linear layers is a badly conditioned function to optimize, and the gradient signal reaching the early layers is weak and noisy.

- **Residual networks improve with depth**: training loss 0.48 → 0.23 → 0.17 and validation accuracy 83.0% → 88.1% → 88.9%. With a shortcut each block only has to learn a residual $F(x) = H(x) - x$; driving F towards 0 gives the identity, so extra depth can never hurt much, and the identity paths carry the gradient unchanged to every layer ($\partial x_L / \partial x_\ell = I + \partial(\sum F)/\partial x_\ell$), which makes the deep network as easy to optimize as a shallow one. ResNet-56 has the lowest training loss of all six models and also the best validation accuracy.
- At **depth 8** the shortcut makes no difference (the two curves overlap): a 3-block network is easy to optimize either way. The benefit of the shortcut grows with depth (+2.5 points at 20 layers, +29.7 points at 56 layers), and since the shortcuts are parameter-free (option A) the comparison isolates the effect of the skip connection itself. The cost of depth is compute: ResNet-56 needs about 8× the time per epoch of ResNet-8 on this GPU.

Task 4: Best CNN with ≤ 500k parameters and ≤ 20 epochs

Design. With only 20 epochs, a shallow-but-wide residual network trains faster than a deep-thin one (Task 3b), so the model is a ResNet with 2 basic blocks per stage (14 weighted layers), widths 26/52/104, projection (1×1 conv) shortcuts at the stage transitions and the last BN of every residual branch initialized to zero (each block starts as the identity). Recipe of the full model:

- SGD with Nesterov momentum 0.9, batch 128, peak LR 0.1, 1 epoch linear warm-up then cosine decay to 0,
- weight decay 5×10^{-4}, label smoothing 0.1,
- augmentation: random flip + crop and cutout (16×16),
- 20 epochs on the 45k training split; model selection only on the 5k validation split.

Ablation. Each variant changes one design choice of the full model and is trained with the same seed and budget. The final model is the variant with the best validation accuracy; only that model is evaluated on the test set, once.

In [18]:
```
T4_ARCH = dict(blocks=(2, 2, 2), widths=(26, 52, 104), shortcut="B",
zero_init_last_bn=True)
```

```
T4_PARAMS = n_params(CifarNet(**T4_ARCH))
print(f"Task 4 model parameters: {T4_PARAMS:,}")
assert T4_PARAMS <= 500_000

VARIANTS_4 = {
    "full model": dict(),
    "no shortcut (plain)": dict(residual=False),
    "no augmentation": dict(aug=None),
    "no cutout (flip+crop only)": dict(aug=flip_crop),
    "no weight decay": dict(wd=0.0),
    "no label smoothing": dict(ls=0.0),
    "AdamW instead of SGD": dict(opt="adamw"),
}


def run_4(name, n_epochs=20):
    cfg = VARIANTS_4[name]
    set_seed()
    model = CifarNet(**{**T4_ARCH, "residual": cfg.get("residual",
True)}).to(device)
    wd = cfg.get("wd", 5e-4)
    if cfg.get("opt") == "adamw":
        opt = torch.optim.AdamW(model.parameters(), lr=2e-3, weight_decay=5e-2)
    else:
        opt = torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9,
nesterov=True, weight_decay=wd)
    spe = math.ceil(len(y_tr) / 128)
    H = fit(model, opt, n_epochs, X_tr, y_tr, bs=128, aug=cfg.get("aug",
compose(flip_crop, cutout)),
            label_smoothing=cfg.get("ls", 0.1), sched=cosine_warmup(opt, n_epochs
* spe, warmup_steps=spe),
            tag=name)
    H["params"] = n_params(model)
    torch.save(model.state_dict(), f"{CKPT}/task4_{name.replace(' ', '_')}.pt")
    return H


R4 = {k: cached(f"task4_{k.replace(' ', '_').replace('(', '').replace(')',
'').replace('+', '_')}",
                lambda k=k: run_4(k, epochs(20))) for k in VARIANTS_4}
```

```
Task 4 model parameters: 460,548
[full model] ep   1/20 loss 1.9915 val_acc 0.4166 (17.3s)
[full model] ep   2/20 loss 1.6357 val_acc 0.4784 (15.0s)
[full model] ep   4/20 loss 1.3462 val_acc 0.6280 (15.0s)
[full model] ep   6/20 loss 1.2196 val_acc 0.6546 (15.0s)
[full model] ep   8/20 loss 1.1243 val_acc 0.7032 (15.0s)
[full model] ep  10/20 loss 1.0550 val_acc 0.7668 (15.0s)
[full model] ep  12/20 loss 0.9978 val_acc 0.7980 (15.0s)
[full model] ep  14/20 loss 0.9393 val_acc 0.8452 (15.0s)
[full model] ep  16/20 loss 0.8796 val_acc 0.8670 (15.0s)
[full model] ep  18/20 loss 0.8405 val_acc 0.8882 (15.0s)
[full model] ep  20/20 loss 0.8228 val_acc 0.8896 (15.0s)
[no shortcut (plain)] ep   1/20 loss 1.8539 val_acc 0.4826 (13.8s)
[no shortcut (plain)] ep   2/20 loss 1.4760 val_acc 0.6194 (13.8s)
[no shortcut (plain)] ep   4/20 loss 1.2455 val_acc 0.7080 (13.8s)
```

```
[no shortcut (plain)] ep   6/20 loss 1.1478 val_acc 0.7112 (13.8s)
[no shortcut (plain)] ep   8/20 loss 1.0859 val_acc 0.6954 (13.8s)
[no shortcut (plain)] ep  10/20 loss 1.0365 val_acc 0.7900 (13.8s)
[no shortcut (plain)] ep  12/20 loss 0.9869 val_acc 0.7856 (13.7s)
[no shortcut (plain)] ep  14/20 loss 0.9331 val_acc 0.8534 (13.7s)
[no shortcut (plain)] ep  16/20 loss 0.8710 val_acc 0.8620 (13.7s)
[no shortcut (plain)] ep  18/20 loss 0.8331 val_acc 0.8872 (13.8s)
[no shortcut (plain)] ep  20/20 loss 0.8146 val_acc 0.8878 (13.7s)
[no augmentation] ep   1/20 loss 1.9002 val_acc 0.4328 (14.9s)
[no augmentation] ep   2/20 loss 1.4235 val_acc 0.6282 (14.9s)
[no augmentation] ep   4/20 loss 1.1175 val_acc 0.6844 (14.9s)
[no augmentation] ep   6/20 loss 0.9519 val_acc 0.7242 (14.9s)
[no augmentation] ep   8/20 loss 0.8567 val_acc 0.7400 (14.9s)
[no augmentation] ep  10/20 loss 0.7733 val_acc 0.7932 (14.9s)
[no augmentation] ep  12/20 loss 0.6854 val_acc 0.8180 (14.9s)
[no augmentation] ep  14/20 loss 0.5943 val_acc 0.8360 (14.9s)
[no augmentation] ep  16/20 loss 0.5414 val_acc 0.8620 (14.9s)
[no augmentation] ep  18/20 loss 0.5288 val_acc 0.8652 (14.9s)
[no augmentation] ep  20/20 loss 0.5262 val_acc 0.8654 (14.9s)
[no cutout (flip+crop only)] ep   1/20 loss 1.9417 val_acc 0.4172 (15.0s)
[no cutout (flip+crop only)] ep   2/20 loss 1.5081 val_acc 0.4660 (15.0s)
[no cutout (flip+crop only)] ep   4/20 loss 1.1879 val_acc 0.6960 (15.0s)
[no cutout (flip+crop only)] ep   6/20 loss 1.0164 val_acc 0.7770 (15.0s)
[no cutout (flip+crop only)] ep   8/20 loss 0.9381 val_acc 0.7700 (15.0s)
[no cutout (flip+crop only)] ep  10/20 loss 0.8808 val_acc 0.8152 (15.0s)
[no cutout (flip+crop only)] ep  12/20 loss 0.8278 val_acc 0.8026 (15.0s)
[no cutout (flip+crop only)] ep  14/20 loss 0.7777 val_acc 0.8624 (14.9s)
[no cutout (flip+crop only)] ep  16/20 loss 0.7232 val_acc 0.8856 (14.9s)
[no cutout (flip+crop only)] ep  18/20 loss 0.6800 val_acc 0.9058 (15.0s)
[no cutout (flip+crop only)] ep  20/20 loss 0.6617 val_acc 0.9078 (15.0s)
[no weight decay] ep   1/20 loss 1.9914 val_acc 0.4216 (15.0s)
[no weight decay] ep   2/20 loss 1.6413 val_acc 0.4904 (15.0s)
[no weight decay] ep   4/20 loss 1.3459 val_acc 0.6280 (15.0s)
[no weight decay] ep   6/20 loss 1.2145 val_acc 0.6412 (15.0s)
[no weight decay] ep   8/20 loss 1.1304 val_acc 0.7362 (15.0s)
[no weight decay] ep  10/20 loss 1.0617 val_acc 0.8034 (15.0s)
[no weight decay] ep  12/20 loss 1.0078 val_acc 0.8258 (15.0s)
[no weight decay] ep  14/20 loss 0.9635 val_acc 0.8430 (15.0s)
[no weight decay] ep  16/20 loss 0.9267 val_acc 0.8510 (15.0s)
[no weight decay] ep  18/20 loss 0.9089 val_acc 0.8602 (15.0s)
[no weight decay] ep  20/20 loss 0.9057 val_acc 0.8648 (15.0s)
[no label smoothing] ep   1/20 loss 1.8861 val_acc 0.4106 (15.0s)
[no label smoothing] ep   2/20 loss 1.4143 val_acc 0.5124 (15.0s)
[no label smoothing] ep   4/20 loss 1.0595 val_acc 0.5838 (15.0s)
[no label smoothing] ep   6/20 loss 0.9089 val_acc 0.6228 (15.0s)
[no label smoothing] ep   8/20 loss 0.7785 val_acc 0.7372 (14.9s)
[no label smoothing] ep  10/20 loss 0.6857 val_acc 0.8006 (14.9s)
[no label smoothing] ep  12/20 loss 0.6113 val_acc 0.7782 (15.0s)
[no label smoothing] ep  14/20 loss 0.5294 val_acc 0.8466 (15.0s)
[no label smoothing] ep  16/20 loss 0.4487 val_acc 0.8726 (14.9s)
[no label smoothing] ep  18/20 loss 0.3929 val_acc 0.8922 (15.0s)
[no label smoothing] ep  20/20 loss 0.3682 val_acc 0.8926 (15.0s)
[AdamW instead of SGD] ep   1/20 loss 1.9790 val_acc 0.4682 (15.0s)
[AdamW instead of SGD] ep   2/20 loss 1.5646 val_acc 0.5468 (15.0s)
[AdamW instead of SGD] ep   4/20 loss 1.3106 val_acc 0.6788 (15.0s)
[AdamW instead of SGD] ep   6/20 loss 1.1752 val_acc 0.7126 (15.0s)
```

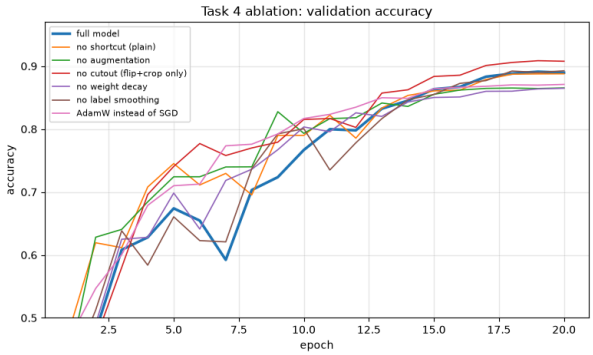
```
[AdamW instead of SGD] ep   8/20 loss 1.0827 val_acc 0.7760 (15.0s)
[AdamW instead of SGD] ep  10/20 loss 1.0207 val_acc 0.8168 (15.0s)
[AdamW instead of SGD] ep  12/20 loss 0.9722 val_acc 0.8348 (15.0s)
[AdamW instead of SGD] ep  14/20 loss 0.9286 val_acc 0.8488 (15.0s)
[AdamW instead of SGD] ep  16/20 loss 0.8940 val_acc 0.8664 (15.0s)
[AdamW instead of SGD] ep  18/20 loss 0.8803 val_acc 0.8702 (15.0s)
[AdamW instead of SGD] ep  20/20 loss 0.8752 val_acc 0.8710 (15.0s)
```

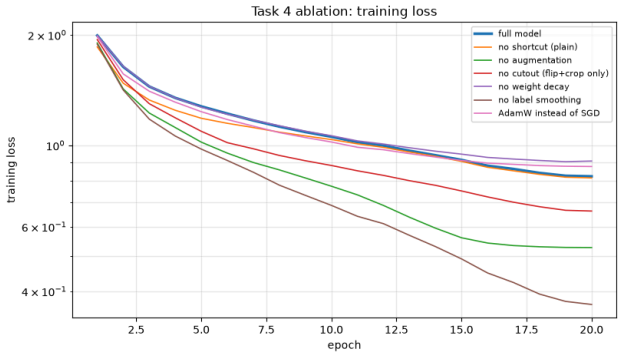
In [19]:
```
base = R4["full model"]["val_acc"][-1]
t4 = pd.DataFrame({k: {"parameters": H["params"], "val acc": H["val_acc"][-1],
                       "change vs full model": H["val_acc"][-1] - base, "best val
acc": max(H["val_acc"]),
                       "final train loss": H["train_loss"][-1]} for k, H in
R4.items()}).T
print("Ablation, final epoch (20), validation split:")
display(t4.style.format({"parameters": "{:,.0f}", "val acc": "{:.2%}", "change vs
full model": "{:+.2%}",
                         "best val acc": "{:.2%}", "final train loss": "{:.4f}"})
       .highlight_max(subset=["val acc"], color="#c6efce"))

fig, ax = plt.subplots(1, 2, figsize=(16, 4.8))
for i, (k, H) in enumerate(R4.items()):
    ep = np.arange(1, len(H["val_acc"]) + 1)
    lw = 2.5 if k == "full model" else 1.2
    ax[0].plot(ep, H["val_acc"], color=cmap(i), lw=lw, label=k)
    ax[1].plot(ep, H["train_loss"], color=cmap(i), lw=lw, label=k)
ax[0].set(xlabel="epoch", ylabel="accuracy", title="Task 4 ablation: validation
accuracy", ylim=(0.5, 0.97))
ax[1].set(xlabel="epoch", ylabel="training loss", title="Task 4 ablation: training
loss", yscale="log")
for a in ax:
    a.grid(alpha=0.3, which="both"); a.legend(fontsize=8)
plt.tight_layout(); savefig("task4_ablation"); plt.show()
```

```
Ablation, final epoch (20), validation split:
```

	parameters	val acc	change vs full model	best val acc	final train loss
full model	460,548	88.96%	+0.00%	89.12%	0.8228
no shortcut (plain)	453,476	88.78%	-0.18%	88.78%	0.8146
no augmentation	460,548	86.54%	-2.42%	86.54%	0.5262
no cutout (flip+crop only)	460,548	90.78%	+1.82%	90.86%	0.6617
no weight decay	460,548	86.48%	-2.48%	86.48%	0.9057
no label smoothing	460,548	89.26%	+0.30%	89.26%	0.3682
AdamW instead of SGD	460,548	87.10%	-1.86%	87.10%	0.8752





Ablation discussion. All variants have the same 20-epoch budget and seed (final-epoch validation accuracy):

variant	val acc	change	final train loss
full model (flip + crop + cutout, wd 5×10^{-4}, LS 0.1, SGD-Nesterov)	88.96%	–	0.823
no shortcut (plain)	88.78%	−0.18	0.815
no augmentation	86.54%	−2.42	0.526
no cutout (flip + crop only)	**90.78%**	**+1.82**	0.662
no weight decay	86.48%	−2.48	0.906
no label smoothing	89.26%	+0.30	0.368
AdamW instead of SGD	87.10%	−1.86	0.875

(With label smoothing 0.1 the training loss cannot go below ≈0.5, so losses are only comparable among the label-smoothed runs.)

- **Augmentation** matters most on the data side: without any augmentation the training loss drops to the label-smoothing floor (memorisation) and validation accuracy falls by 2.4 points. But **cutout on top of flip + crop hurts with only 20 epochs** (−1.8 points vs flip + crop only): the training loss is still high (0.82 vs 0.66), i.e. the model is *under*-fitted. Cutout is a strong regulariser that pays off in long (200-epoch) schedules, not in a 20-epoch budget, the same capacity/budget trade-off as "all combined" in Task 3(a).
- **Weight decay** is essential (−2.5 points without it). Interestingly its removal makes the *training* loss higher, not lower: with batch norm the convolutions are scale-invariant, and without weight decay their norms grow, which shrinks the effective learning rate $\eta/\|w\|^2$ and slows optimization. So in BN networks weight decay acts mainly as an effective-learning-rate control.
- **Optimizer:** AdamW (lr 2×10^{-3}, decoupled wd 0.05, same schedule) is 1.9 points worse than SGD with Nesterov momentum, in line with Task 2(c) where tuned momentum-SGD also edged out Adam on CIFAR.
- **Shortcut:** at this depth (14 layers) removing the shortcuts costs only 0.2 points, consistent with Task 3(b) where the gap between plain and residual networks is small for shallow networks and only becomes large with depth. It is kept because it makes optimization robust and costs no time.

- **Label smoothing** changes accuracy by only +0.3 points (within run-to-run noise of a few tenths of a percent).

Following the selection rule (best final validation accuracy), the **final model is the residual network with flip + crop (no cutout), weight decay 5×10^{-4}, label smoothing 0.1 and SGD-Nesterov** (460,548 parameters, 20 epochs, 90.78% validation accuracy).

Final model: single evaluation on the test set

The variant with the highest final validation accuracy is selected and its saved weights are evaluated on the 10k CIFAR-10 test images. The result is cached, so the test set is used exactly once.

In [20]:
```
FINAL = max(R4, key=lambda k: R4[k]["val_acc"][-1])
print("selected on validation:", FINAL, f"(val acc {R4[FINAL]['val_acc']
[-1]:.4f})")

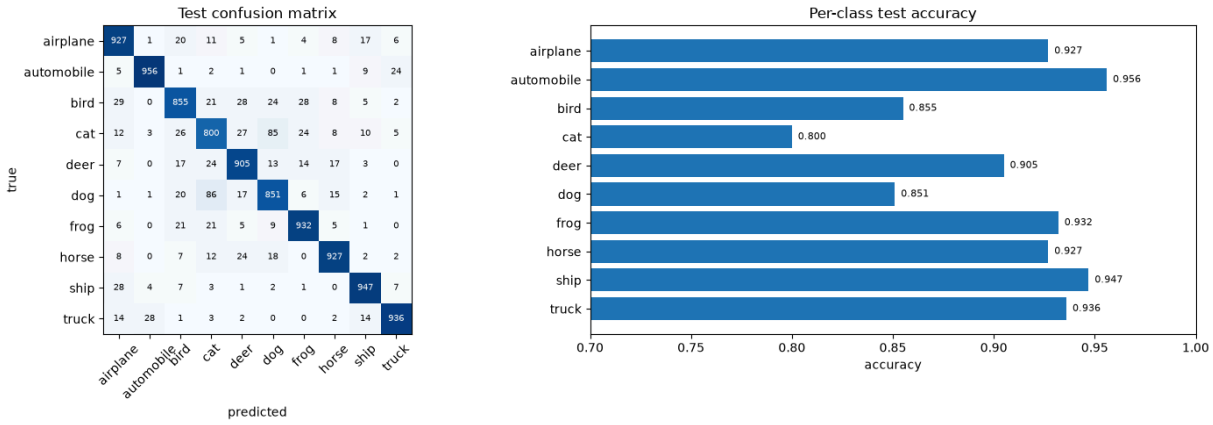

def test_once():
    cfg = VARIANTS_4[FINAL]
    model = CifarNet(**{**T4_ARCH, "residual": cfg.get("residual",
True)}).to(device)
    model.load_state_dict(torch.load(f"{CKPT}/task4_{FINAL.replace(' ', '_')}.pt",
map_location=device))
    model.eval()
    preds = []
    with torch.no_grad():
        for i in range(0, len(y_te), 1000):
            with torch.autocast(device.type, dtype=torch.float16, enabled=AMP):
                preds.append(model(X_te[i:i + 1000].to(device)).argmax(1).cpu())
    preds = torch.cat(preds)
    cm = torch.zeros(10, 10, dtype=torch.long)
    for t, p in zip(y_te, preds):
        cm[t, p] += 1
    return dict(model=FINAL, test_acc=(preds == y_te).float().mean().item(),
confusion=cm.tolist())


T4_TEST = cached("task4_test", test_once)
print(f"Final model: {T4_TEST['model']}  |  parameters {T4_PARAMS:,}  |  TEST
ACCURACY = {T4_TEST['test_acc']:.4f}")

cm = np.array(T4_TEST["confusion"])
fig, ax = plt.subplots(1, 2, figsize=(15, 5))
im = ax[0].imshow(cm / cm.sum(1, keepdims=True), cmap="Blues", vmin=0, vmax=1)
for i in range(10):
    for j in range(10):
        ax[0].text(j, i, cm[i, j], ha="center", va="center", fontsize=7,
color="white" if i == j else "black")
ax[0].set_xticks(range(10)); ax[0].set_xticklabels(CLASSES, rotation=45);
ax[0].set_yticks(range(10))
ax[0].set_yticklabels(CLASSES); ax[0].set(xlabel="predicted", ylabel="true",
```

```
title="Test confusion matrix")
per_class = cm.diagonal() / cm.sum(1)
ax[1].barh(CLASSES, per_class, color="tab:blue"); ax[1].set_xlim(0.7, 1.0)
for i, v in enumerate(per_class):
    ax[1].text(v + 0.003, i, f"{v:.3f}", va="center", fontsize=8)
ax[1].set(title="Per-class test accuracy", xlabel="accuracy");
ax[1].invert_yaxis()
plt.tight_layout(); savefig("task4_test"); plt.show()
```

```
selected on validation: no cutout (flip+crop only) (val acc 0.9078)
Final model: no cutout (flip+crop only)  |  parameters 460,548  |  TEST ACCURACY = 0.9036
```



Test result. The selected model reaches **90.36% test accuracy** (evaluated once), with 460,548 parameters (< 500k) and 20 training epochs. This is only 0.4 points below its validation accuracy (90.78%), so selecting among seven variants on the 5k validation split introduced little optimistic bias. The per-class results show where the remaining errors are: vehicles are easiest (automobile 95.6%, ship 94.7%, truck 93.6%), while the animal classes with similar shapes and textures are hardest: **cat (80.0%) and dog (85.1%) are mostly confused with each other** (85 cats predicted as dog and 86 dogs as cat), and bird (85.5%) is confused with airplane, deer and frog. The vehicle errors are also semantically sensible (truck ↔ automobile, ship ↔ airplane, which share sky/sea backgrounds).

Summary

task	key result
1	Manual backprop matches autograd to ~10^{-16} and finite differences to ~10^{-9}; MLP: 52.0% val. accuracy with strong overfitting; sigmoid gradients vanish ~$10^{4}\times$ over 10 layers, ReLU + He init keeps them constant.
2	Own SGD-momentum/Adam match `torch.optim` exactly; momentum and Adam fix the zig-zag of SGD on an ill-conditioned quadratic; Adam is best at lr 10^{-3} but fails at $\geq 10^{-2}$, momentum-SGD is the most robust (72.3% at lr 0.1).
3a	On 5k images flip + crop is the best single regulariser (68.2% → 73.9%); combining all regularisers minimises the gap (9 points) but underfits.
3b	Plain nets degrade with depth (56 layers: train loss 1.15, 59.3%), residual nets improve (ResNet-56: 0.17, 88.9%): shortcuts solve an optimization problem, not overfitting.

task	key result
4	460k-parameter ResNet, 20 epochs: **90.36% test accuracy**; weight decay and augmentation matter most, cutout hurts at this short budget.